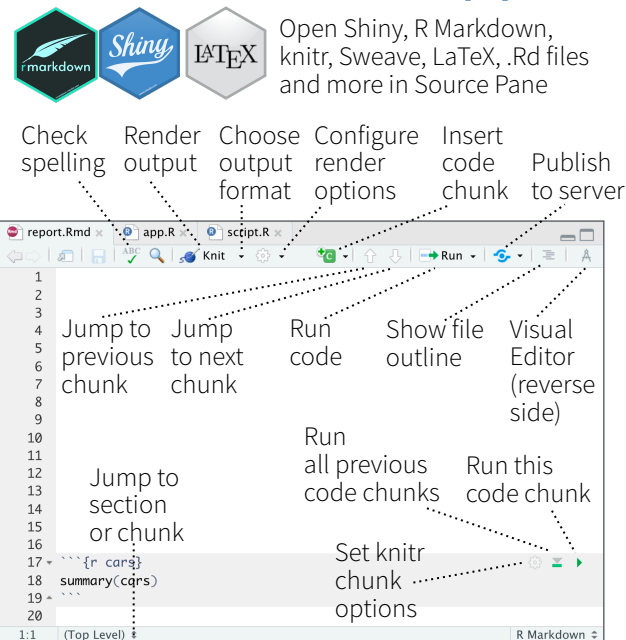


RStudio IDE :: CHEATSHEET

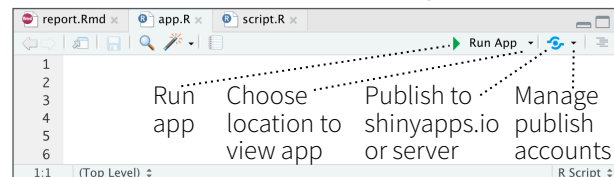


Documents and Apps

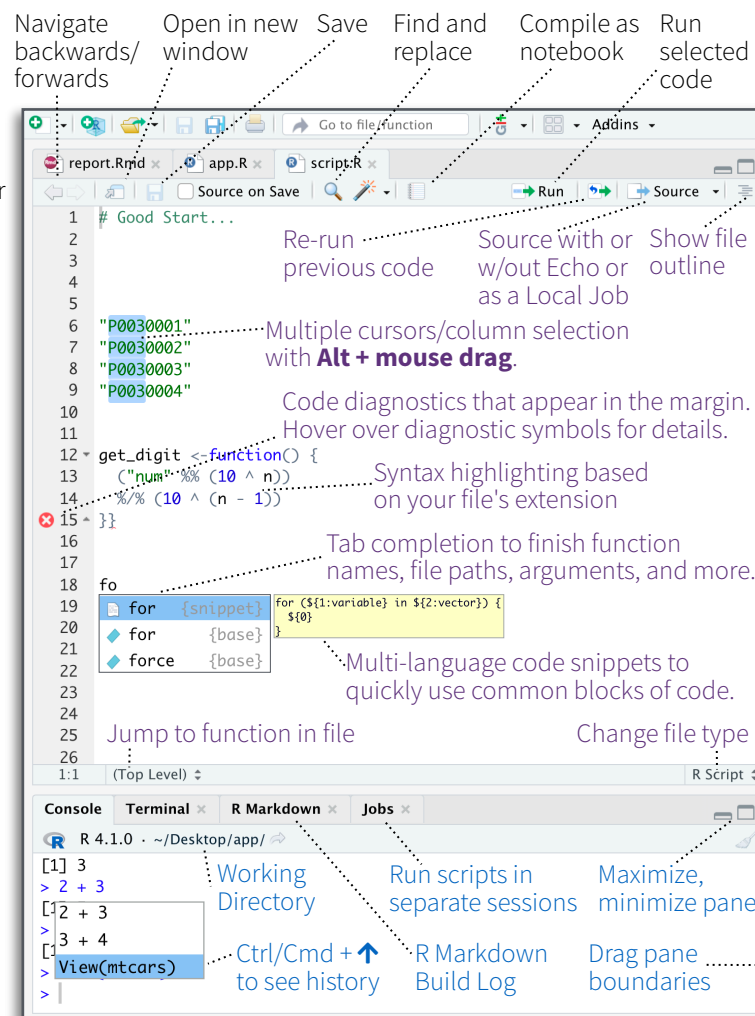


Access markdown guide at **Help > Markdown Quick Reference**
See reverse side for more on **Visual Editor**

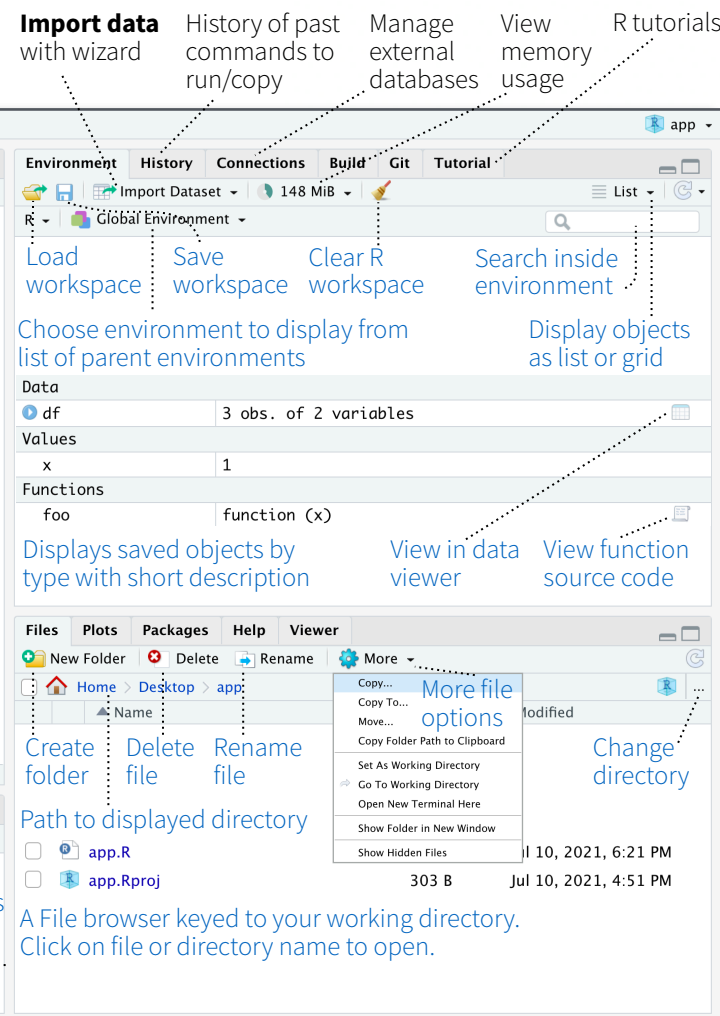
RStudio recognizes that files named **app.R**, **server.R**, **ui.R**, and **global.R** belong to a shiny app



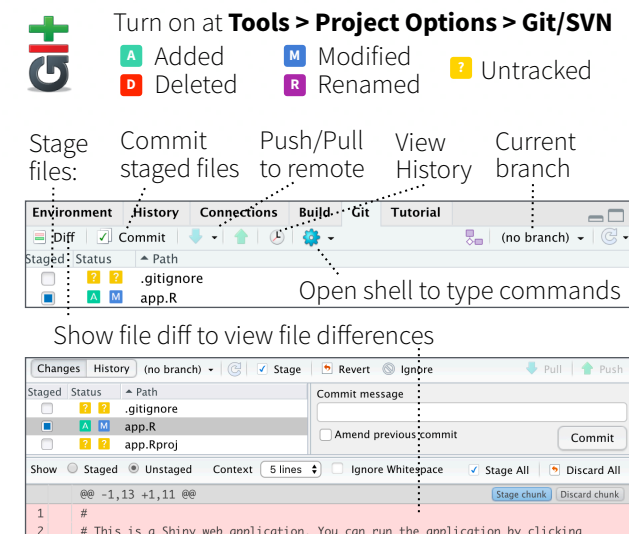
Source Editor



Tab Panes

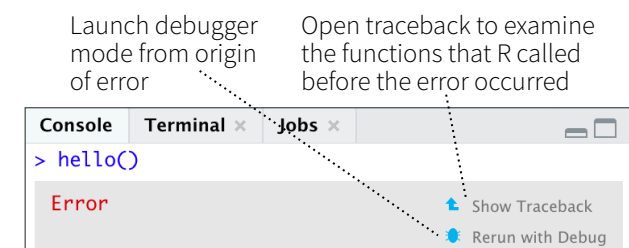


Version Control



Debug Mode

Use **debug()**, **browser()**, or a breakpoint and execute your code to open the debugger mode.

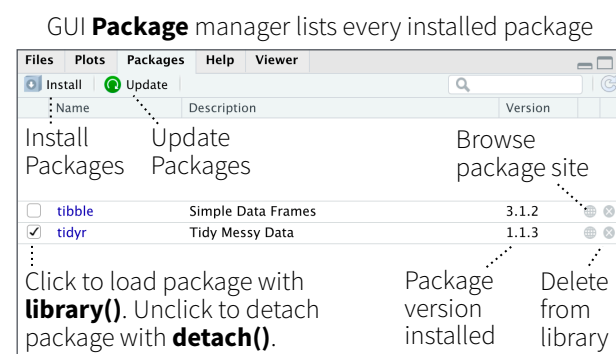
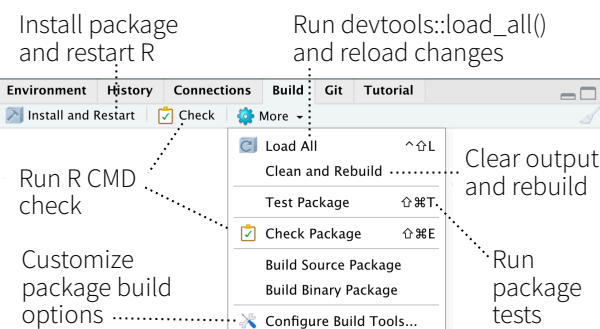


Package Development

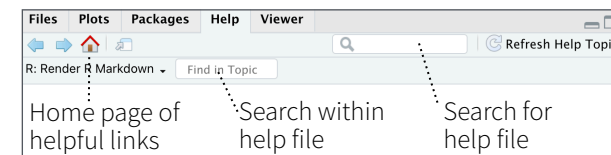
Create a new package with **File > New Project > New Directory > R Package**
Enable roxygen documentation with **Tools > Project Options > Build Tools**

Roxygen guide at **Help > Roxygen Quick Reference**

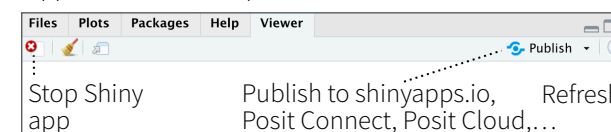
See package information in the **Build Tab**



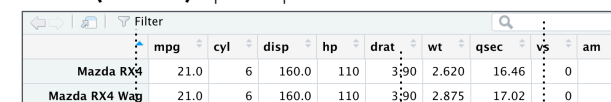
RStudio opens documentation in a dedicated **Help** pane



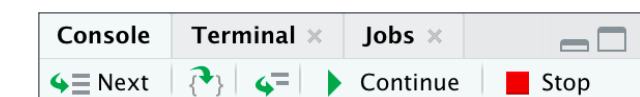
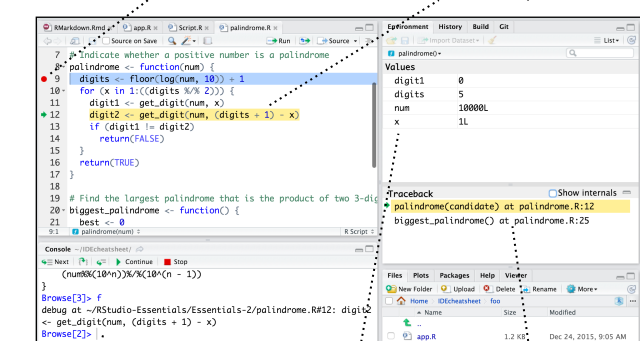
Viewer pane displays HTML content, such as Shiny apps, RMarkdown reports, and interactive visualizations



View(<data>) opens spreadsheet like view of data set



Click next to line number to add/remove a breakpoint. Highlighted line shows where execution has paused



Keyboard Shortcuts

[RUN CODE](#)

Search command history	Ctrl+↑	Cmd+↑
Interrupt current command	Esc	Esc
Clear console	Ctrl+L	Ctrl+L

NAVIGATE CODE

Go to File/Function	Ctrl+.	Ctrl+.
---------------------	--------	--------

WRITE CODE

Attempt completion Tab or Ctrl+Space Tab or Ctrl+Space

Insert <- (assignment operator)	Alt+-	Option+-
Insert > or %>% (pipe operator)	Ctrl+Shift+M	Cmd+Shift+M
(Un)Comment selection	Ctrl+Shift+C	Cmd+Shift+C

MAKE PACKAGES

Load All (devtools)	Ctrl+Shift+L	Cmd+Shift+L
Test Package (Desktop)	Ctrl+Shift+T	Cmd+Shift+T
Document Package	Ctrl+Shift+D	Cmd+Shift+D

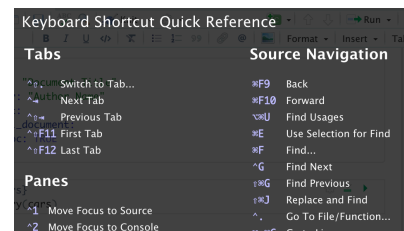
DOCUMENTS AND APPS

Knit/Render Document (knitr)	Ctrl+Shift+K	Cmd+Shift+K
Insert chunk (Sweave & Knitr)	Ctrl+Alt+I	Cmd+Option+I
Run from start to current line	Ctrl+Alt+B	Cmd+Option+B

MORE KEYBOARD SHORTCUTS

Keyboard Shortcuts Help	Alt+Shift+K	Option+Shift+K
Show Command Palette	Ctrl+Shift+P	Cmd+Shift+P

View the Keyboard Shortcut Quick Reference with **Tools > Keyboard Shortcuts** or **Alt/Option + Shift + K**



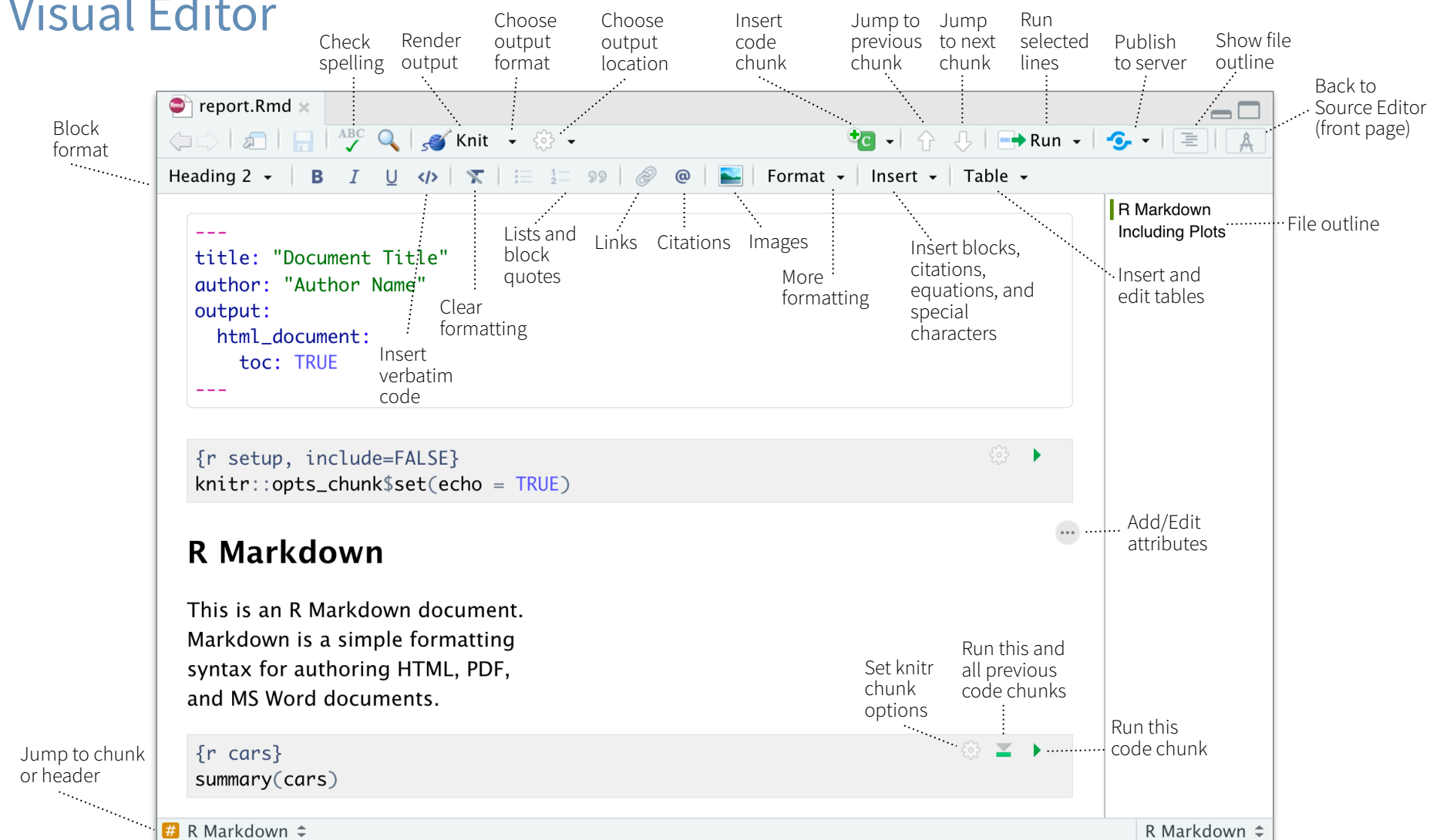
Search for keyboard shortcuts with **Tools > Show Command Palette** or **Ctrl/Cmd + Shift + P**.

History Send Command to Console

Create a New R Script Ctrl Shift

Create a new R Markdown document

Visual Editor



Posit Workbench

WHY POSIT WORKBENCH?

Extend the open source server with a commercial license, support, and more:

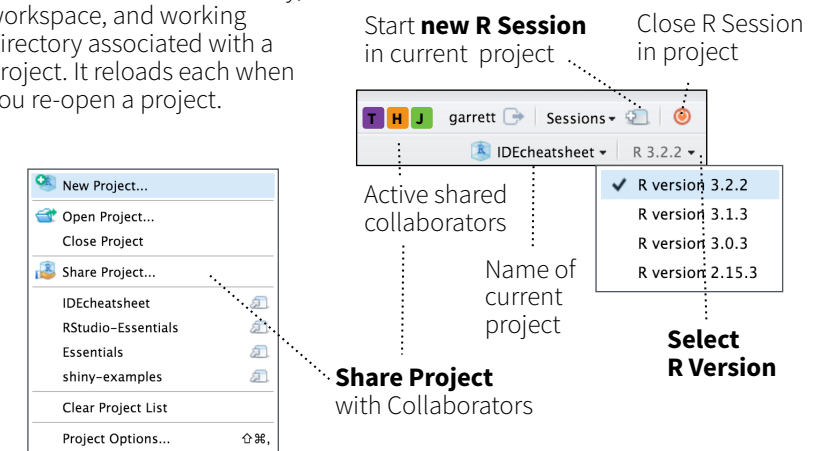
- open and run multiple R sessions at once
- tune your resources to improve performance
- administrative tools for managing user sessions
- collaborate real-time with others in shared projects
- switch easily from one version of R to a different version
- integrate with your authentication, authorization, and audit practices
- work in the RStudio IDE, JupyterLab, Jupyter Notebooks, or VS Code

Download a free 45 day evaluation at
posit.co/products/enterprise/workbench/

Share Projects

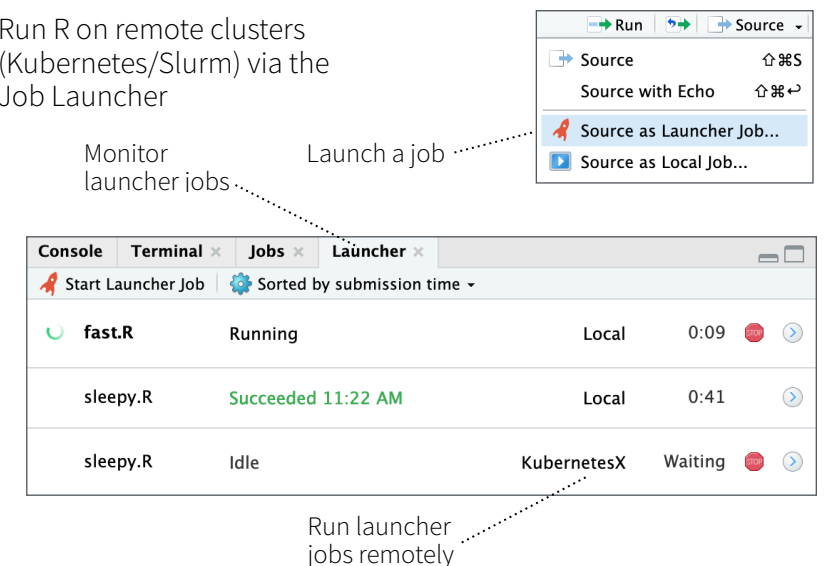
File > New Project

RStudio saves the call history, workspace, and working directory associated with a project. It reloads each when you re-open a project.



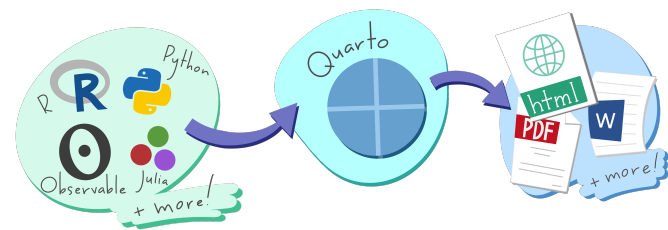
Run Remote Jobs

Run R on remote clusters (Kubernetes/Slurm) via the Job Launcher



Publish and Share with Quarto : : CHEATSHEET

quarto



Author

WRITE AND CODE IN PLAIN TEXT

Author documents as .qmd files or Jupyter notebooks. Write in a rich Markdown syntax.

Render

GENERATE DOCUMENTS, PRESENTATIONS AND MORE

Produce HTML, PDF, MS Word reveal.js, MS Powerpoint, Beamer Websites, blogs, books...

Publish

SHARE YOUR WORK WITH THE WORLD

Quickly deploy to GitHub Pages, Netlify, Quarto Pub, Posit Cloud, or Posit Connect

Author

SOURCE FILE: hello.qmd

```
---
title: "Hello, Penguins"
format: html
execute:
  echo: false
---

## Meet the penguins

The `penguins` data contain
from three islands in the P

The three species of penguin have quite distinct
distributions of physical dimensions (@fig-penguins).

```{r}
#| label: fig-penguins
#| fig-cap: "Dimensions of penguins a
#| warning: false
library(tidyverse, quietly = TRUE)
library(palmerpenguins)
penguins |>
 ggplot(aes(x = flipper_length_mm, y = bill_length_mm)) +
 geom_point(aes(color = species)) +
 scale_color_manual(
 values = c("darkorange", "purple", "cyan4")) +
```

Set format(s) and options  
Use YAML Syntax

## Write with **Markdown**  
RStudio: Help > Markdown Quick Reference  
Use Visual Editor

Include code  
R, Python, Julia, Observable,  
or any language with a  
Jupyter kernel

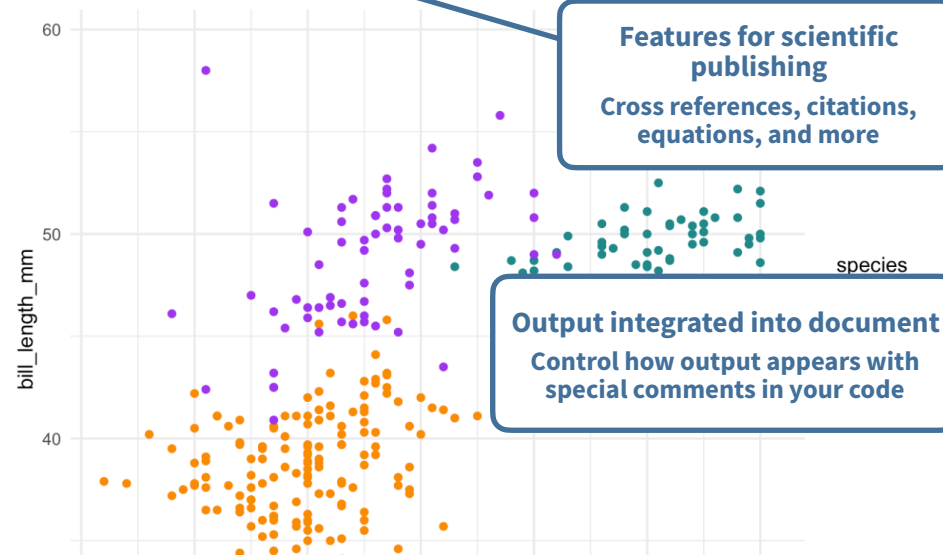
## Render

### RENDERED OUTPUT: hello.html

### Hello, Penguins

#### Meet the penguins

The three species of penguins have quite distinct distributions of physical dimensions ([Figure 1](#)).



Features for scientific  
publishing  
Cross references, citations,  
equations, and more

Output integrated into document  
Control how output appears with  
special comments in your code

### USE A TOOL WITH A RICH EDITING EXPERIENCE

RStudio Positron VS Code + Quarto extension

Run code cells as you write

Render with a button or keyboard shortcut

Edit Quarto documents with a Visual Editor

### OR ANY TEXT EDITOR

Quarto documents (.qmd) can be edited in any tool that edits text.

Apply formatting in  
Visual Editor. Saved  
as Markdown in  
source.

Insert elements like  
code cells, cross  
references, and  
more.

Normal B I </> :≡ 1/3 ↺ 🖨 Format Insert Table

Save, then render to **preview** the document output.

Terminal  
quarto **preview** hello.qmd

Use **Render** button

Use **Preview** button

The resulting HTML/PDF/MS Word/etc. document will be created and saved in the same directory as the source .qmd file.

### BEHIND THE SCENES

When you render a document, Quarto:

1. Runs the code and embeds results and text into an .md file with: **Knitr**, if any {r} cells or, **Jupyter**, if any other cells.
2. Converts the .md file into the output format with Pandoc.

### GET QUARTO

<https://quarto.org/docs/download/>

Or use version **bundled with Positron or RStudio**

### GET STARTED

<https://quarto.org/docs/get-started/>

## Publish

### Terminal

quarto **publish** {venue} hello.qmd

{venue}: quarto-pub, connect, gh-pages, netlify, confluence

Quarto Pub

Free publishing service for Quarto content.

posit Connect

Org-hosted, control access, schedule updates.

Use **Publish** button

Use **Posit Publisher** extension

## Quarto Projects

### CREATE WEBSITES, BOOKS, AND MORE

A directory of Quarto documents + a configuration file (\_quarto.yml)

See examples at <https://quarto.org/docs/gallery/>

Get started from the command line:

### Terminal

quarto **create project** {type}

{type}: default, website, blog, book, confluence, manuscript

Use **File > New Project**

Use **Quarto: Create Project** command

Artwork from "Hello, Quarto" keynote by Julia Lowndes and Mine Çetinkaya-Rundel, presented at RStudio Conference 2022. Illustrated by Allison Horst.

# Include Code

## CODE CELLS

Code cells start with ````{language}` and end with `````.

   Use **Insert Code Chunk/Cell**

```
```{r}
#| label: chunk-id
library(tidyverse)
```

```{python}
#| label: chunk-id
import pandas as pd
```
```

Other languages: `{julia}`, `{ojs}`

Add code cell options with `#|` comments.

Cell options control **execution**, figures, tables, layout and more. See them all at:  
<https://quarto.org/docs/reference/cells>

## EXECUTION OPTIONS

### OPTION DEFAULT EFFECTS

|                |       |                                                                     |
|----------------|-------|---------------------------------------------------------------------|
| <b>echo</b>    | true  | false: hide code<br>fenced: include code cell syntax                |
| <b>eval</b>    | true  | false: don't run code                                               |
| <b>include</b> | true  | false: don't include code or results                                |
| <b>output</b>  | true  | false: don't include results<br>asis: treat results as raw markdown |
| <b>warning</b> | true  | false: don't include warnings in output                             |
| <b>error</b>   | false | true: include error in output and<br>continue with render           |

Set execution options at the **cell level**:

```
```{r}
#| echo: false
```

```{python}
#| echo: false
```
```

Or, **globally** in the YAML header with the **execute** option:

```

execute:
 echo: false

```

**Set options in code cells with `#|` comments and YAML syntax:**  
key: value

## INLINE CODE

Use computed values directly in text sections. Code is evaluated at render and results appear as text.

**KNITR** Value is ``r 2 + 2``. **JUPYTER** Value is ``{python} 2 + 2``. **OUTPUT** Value is 4.

# Set Format and Options

## SET FORMAT OPTIONS

```

title: "My Document"
format:
 html:
 code-fold: true
 toc: true

```

Indent format 2 spaces

Common formats: **html**, **pdf**, **docx**, **odt**, **rtf**, **gfm**, **pptx**, **revealjs**, **beamer**

Render **all** formats:

```
Terminal
quarto render hello.qmd
```

Render a **specific** format:

```
Terminal
quarto render hello.qmd --to pdf
```

## MULTIPLE FORMATS

```

title: "My Document"
toc: true
format:
 html:
 code-fold: true
 pdf: default

```

Top-level options apply to all formats

## OPTION

Nav

Style

LaTeX

Code

Figures

Also use in code cells

✓

✓

✓

Knitr

## TABLES

### MARKDOWN

```
object	radius
Sun	696000
Earth	6371

: CAPTION {#tbl-LABEL}
```

   **Insert Table in Visual Editor**

## CITATIONS

1. Add a bibliography **file** to the YAML header:

```

bibliography: references.bib

```

2. Add citations: `[@citation]`, or `@citation`

   Use **Insert Citations** dialog in the **Visual Editor**

Build your bibliography file from your Zotero library, DOI, Crossref, DataCite, or PubMed

**COMPUTATION** Output a Markdown table or an HTML table from your code

### KNITR

Use `knitr::kable()` to produce Markdown:

```
```{r}
#| label: tbl-LABEL
#| tbl-cap: CAPTION
knitr::kable(head(cars))
```
```

Also see the R packages: `gt`, `flextable`, `kableExtra`.

**JUPYTER** Add `Markdown()` to Markdown output:

```
```{python}
#| label: tbl-LABEL
#| tbl-cap: CAPTION
import pandas as pd, tabulate
from IPython.display import Markdown
df = pd.DataFrame({"A": [1, 2],
                  "B": [1, 2]})
Markdown(df.to_markdown(index=False))
```
```

## CALLOUTS

```
::: {.callout-tip}
Title
```

Text  
:::

Instead of **tip** use one of:  
note, caution, warning, or important.

 **note**  **warning**  
 **caution**  **important**

## SHORTCODES

```
{{< include _file.qmd >}}
{{< embed file.ipynb#id >}}
{{< video video.mp4 >}}
```

# Add Content

## FIGURES

### MARKDOWN

```
![CAP](image.png){#fig-LABEL fig-alt="ALT"}
```

### COMPUTATION

```
```{python}
#| label: fig-LABEL
#| fig-cap: CAP
#| fig-alt: ALT
{{ plot code here }}
```
```

Or `{r}`

## CROSS REFERENCES

- Add labels**  
Code cell: add option `label: prefix-LABEL`  
Markdown: add attribute `#prefix-LABEL`
- Add references** @prefix-LABEL, e.g.

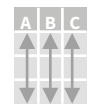
You can see in `@fig-scatterplot`, that...

| Prefix | Renderers | Prefix | Renderers  |
|--------|-----------|--------|------------|
| fig-   | Figure 1  | eq-    | Equation 1 |
| tbl-   | Table 1   | sec-   | Section 1  |

# Data transformation with dplyr : : CHEATSHEET



**dplyr** functions work with pipes and expect **tidy data**. In tidy data:



&



**pipes**

Each **variable** is in its own **column**

Each **observation**, or **case**, is in its own **row**

**x** > **f(y)** becomes **f(x, y)**

## Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

**summary function**



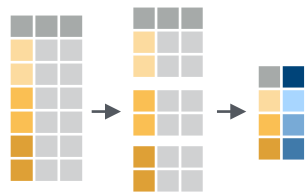
**summarize(.data, ...)**  
Compute table of summaries.  
`mtcars |> summarize(avg = mean(mpg))`



**count(.data, ..., wt = NULL, sort = FALSE, name = NULL)** Count number of rows in each group defined by the variables in ... Also **tally()**, **add\_count()**, **add\_tally()**.  
`mtcars |> count(cyl)`

## Group Cases

Use **group\_by(.data, ..., .add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



`mtcars |> group_by(cyl) |> summarize(avg = mean(mpg))`

Alternate grouping syntax with **.by** as an argument:

`mtcars |> summarize(avg = mean(mpg), .by = cyl)`

Use **rowwise(.data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.



`starwars |> rowwise() |> mutate(film_count = length(films))`

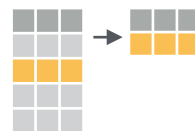
**ungroup(x, ...)** Returns ungrouped copy of table.

`g_mtcars <- mtcars |> group_by(cyl)`  
`ungroup(g_mtcars)`

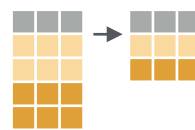
## Manipulate Cases

### EXTRACT CASES

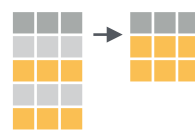
Row functions return a subset of rows as a new table.



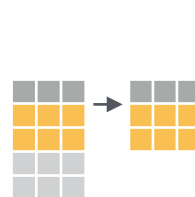
**filter(.data, ..., .preserve = FALSE)** Extract rows that meet logical criteria.  
`mtcars |> filter(mpg > 20)`



**distinct(.data, ..., .keep\_all = FALSE)** Remove rows with duplicate values.  
`mtcars |> distinct(gear)`



**slice(.data, ..., .preserve = FALSE)** Select rows by position.  
`mtcars |> slice(10:15)`



**slice\_sample(.data, ..., n, prop, weight\_by = NULL, replace = FALSE)** Randomly select rows. Use `n` to select a number of rows and `prop` to select a fraction of rows.  
`mtcars |> slice_sample(n = 5, replace = TRUE)`

**slice\_min(.data, order\_by, ..., n, prop, with\_ties = TRUE)** and **slice\_max()** Select rows with the lowest and highest values.  
`mtcars |> slice_min(mpg, prop = 0.25)`

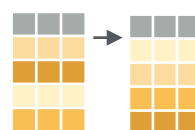
**slice\_head(.data, ..., n, prop)** and **slice\_tail()** Select the first or last rows.  
`mtcars |> slice_head(n = 5)`

### Logical and boolean operators to use with filter()

|                 |                   |                    |                       |                   |                    |                    |
|-----------------|-------------------|--------------------|-----------------------|-------------------|--------------------|--------------------|
| <code>==</code> | <code>&lt;</code> | <code>&lt;=</code> | <code>is.na()</code>  | <code>%in%</code> | <code> </code>     | <code>xor()</code> |
| <code>!=</code> | <code>&gt;</code> | <code>&gt;=</code> | <code>!is.na()</code> | <code>!</code>    | <code>&amp;</code> |                    |

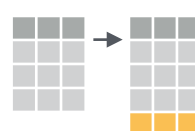
See **?base::Logic** and **?Comparison** for help.

### ARRANGE CASES



**arrange(.data, ..., .by\_group = FALSE)** Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.  
`mtcars |> arrange(mpg)`  
`mtcars |> arrange(desc(mpg))`

### ADD CASES



**add\_row(.data, ..., .before = NULL, .after = NULL)** Add one or more rows to a table.  
`cars |> add_row(speed = 1, dist = 1)`

## Manipulate Variables

### EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



**pull(.data, var = -1, name = NULL, ...)** Extract column values as a vector, by name or index.  
`mtcars |> pull(wt)`



**select(.data, ...)** Extract columns as a table.  
`mtcars |> select(mpg, wt)`



**relocate(.data, ..., .before = NULL, .after = NULL)** Move columns to new position.  
`mtcars |> relocate(mpg, cyl, .after = last_col())`

### Use these helpers with select() and across()

e.g. `mtcars |> select(mpg:cyl)`

|                           |                                       |                                     |
|---------------------------|---------------------------------------|-------------------------------------|
| <b>contains(match)</b>    | <b>num_range(prefix, range)</b>       | <b>;</b> e.g., <code>mpg:cyl</code> |
| <b>ends_with(match)</b>   | <b>all_of(x)/any_of(x, ..., vars)</b> | <b>!</b> e.g., <code>!gear</code>   |
| <b>starts_with(match)</b> | <b>matches(match)</b>                 | <b>everything()</b>                 |

### MANIPULATE MULTIPLE VARIABLES AT ONCE

`df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))`



**across(.cols, .funs, ..., .names = NULL)** Summarize or mutate multiple columns in the same way.  
`df |> summarize(across(everything(), mean))`



**c\_across(.cols)** Compute across columns in row-wise data.  
`df |> rowwise() |> mutate(x_total = sum(c_across(1:2)))`

### MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

**vectorized function**



**mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL)** Compute new column(s). Also **add\_column()**.  
`mtcars |> mutate(gpm = 1 / mpg)`  
`mtcars |> mutate(gpm = 1 / mpg, .keep = "none")`



**rename(.data, ...)** Rename columns. Use **rename\_with()** to rename with a function.  
`mtcars |> rename(miles_per_gallon = mpg)`



# Vectorized Functions

## TO USE WITH MUTATE ()

**mutate()** applies vectorized functions to columns to create new columns. Vectorized functions take vectors as input and return vectors of the same length as output.

**vectorized function**

## OFFSET

`dplyr::lag()` - offset elements by 1  
`dplyr::lead()` - offset elements by -1

## CUMULATIVE AGGREGATE

`dplyr::cumall()` - cumulative all()  
`dplyr::cumany()` - cumulative any()  
`dplyr::cummax()` - cumulative max()  
`dplyr::cummean()` - cumulative mean()  
`dplyr::cummin()` - cumulative min()  
`dplyr::cumprod()` - cumulative prod()  
`dplyr::cumsum()` - cumulative sum()

## RANKING

`dplyr::cume_dist()` - proportion of all values <=  
`dplyr::dense_rank()` - rank w ties = min, no gaps  
`dplyr::min_rank()` - rank with ties = min  
`dplyr::ntile()` - bins into n bins  
`dplyr::percent_rank()` - min\_rank scaled to [0,1]  
`dplyr::row_number()` - rank with ties = "first"

## MATH

`+`, `-`, `*`, `/`, `^`, `%/%`, `%%` - arithmetic ops  
`log()`, `log2()`, `log10()` - logs  
`<`, `<=`, `>`, `>=`, `!=`, `==` - logical comparisons  
`dplyr::between()` - `x >= left & x <= right`  
`dplyr::near()` - safe == for floating point numbers

## MISCELLANEOUS

`dplyr::case_when()` - multi-case if\_else()  

```
starwars |>
 mutate(type = case_when(
 height > 200 | mass > 200 ~ "large",
 species == "Droid" ~ "robot",
 TRUE ~ "other"))
```

`dplyr::coalesce()` - first non-NA values by element across a set of vectors  
`dplyr::if_else()` - element-wise if() + else()  
`dplyr::na_if()` - replace specific values with NA  
`dplyr::pmax()` - element-wise max()  
`dplyr::pmin()` - element-wise min()

# Summary Functions

## TO USE WITH SUMMARIZE ()

**summarize()** applies summary functions to columns to create a new table. Summary functions take vectors as input and return single values as output.

**summary function**

## COUNT

`dplyr::n()` - number of values/rows  
`dplyr::n_distinct()` - # of uniques  
`sum(!is.na())` - # of non-NAs

## POSITION

`mean()` - mean, also `mean(!is.na())`  
`median()` - median

## LOGICAL

`mean()` - proportion of TRUEs  
`sum()` - # of TRUEs

## ORDER

`dplyr::first()` - first value  
`dplyr::last()` - last value  
`dplyr::nth()` - value in nth location of vector

## RANK

`quantile()` - nth quantile  
`min()` - minimum value  
`max()` - maximum value

## SPREAD

`IQR()` - Inter-Quartile Range  
`mad()` - median absolute deviation  
`sd()` - standard deviation  
`var()` - variance

# Row Names

Tidy data does not use rownames, which store a variable outside of the columns. To work with the rownames, first move them into a column.

`tibble::rownames_to_column()`  
Move row names into col.  

```
a <- mtcars |>
rownames_to_column(var = "C")
```

`tibble::column_to_rownames()`  
Move col into row names.  

```
a |> column_to_rownames(var = "C")
```

Also `tibble::has_rownames()` and `tibble::remove_rownames()`.

# Combine Tables

## COMBINE VARIABLES

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |
| c | v | 3 |

 + 

| E | F | G |
|---|---|---|
| a | t | 3 |
| b | u | 2 |
| d | w | 1 |

 = 

| A | B | C | E | F | G |
|---|---|---|---|---|---|
| a | t | 1 | a | t | 3 |
| b | u | 2 | b | u | 2 |
| c | v | 3 | d | w | 1 |

**bind\_cols(..., .name\_repair)** Returns tables placed side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.

## RELATIONAL DATA

Use a "**Mutating Join**" to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

| A | B | C | D  |
|---|---|---|----|
| a | t | 1 | 3  |
| b | u | 2 | 2  |
| c | v | 3 | NA |

**left\_join**(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na\_matches = "na") Join matching values from y to x.

| A | B | C  | D |
|---|---|----|---|
| a | t | 1  | 3 |
| b | u | 2  | 2 |
| d | w | NA | 1 |

**right\_join**(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na\_matches = "na") Join matching values from x to y.

| A | B | C | D |
|---|---|---|---|
| a | t | 1 | 3 |
| b | u | 2 | 2 |

**inner\_join**(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na\_matches = "na") Join data. Retain only rows with matches.

| A | B | C  | D  |
|---|---|----|----|
| a | t | 1  | 3  |
| b | u | 2  | 2  |
| c | v | 3  | NA |
| d | w | NA | 1  |

**full\_join**(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na\_matches = "na") Join data. Retain all values, all rows.

## COLUMN MATCHING FOR JOINS

| A | B.x | C | B.y | D  |
|---|-----|---|-----|----|
| a | t   | 1 | t   | 3  |
| b | u   | 2 | u   | 2  |
| c | v   | 3 | NA  | NA |

 Use **by = c("col1", "col2", ...)** to specify one or more common columns to match on.  
`left_join(x, y, by = "A")`

| A.x | B.x | C | A.y | B.y |
|-----|-----|---|-----|-----|
| a   | t   | 1 | d   | w   |
| b   | u   | 2 | b   | u   |
| c   | v   | 3 | a   | t   |

 Use a named vector, **by = c("col1" = "col2")**, to match on columns that have different names in each table.  
`left_join(x, y, by = c("C" = "D"))`

| A1 | B1 | C | A2 | B2 |
|----|----|---|----|----|
| a  | t  | 1 | d  | w  |
| b  | u  | 2 | b  | u  |
| c  | v  | 3 | a  | t  |

 Use **suffix** to specify the suffix to give to unmatched columns that have the same name in both tables.  
`left_join(x, y, by = c("C" = "D"), suffix = c("1", "2"))`

## COMBINE CASES

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |

 + 

| A | B | C |
|---|---|---|
| c | v | 3 |
| d | w | 4 |

 = 

| D | A | B | C |
|---|---|---|---|
| x | a | t | 1 |
| x | b | u | 2 |
| y | c | v | 3 |
| y | d | w | 4 |

**bind\_rows(..., .id = NULL)** Returns tables one on top of the other as a single table. Set .id to a column name to add a column of the original table names (as pictured).

Use a "**Filtering Join**" to filter one table against the rows of another.

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |
| c | v | 3 |

 + 

| A | B | D |
|---|---|---|
| a | t | 3 |
| b | u | 2 |
| d | w | 1 |

 =

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |

**semi\_join**(x, y, by = NULL, copy = FALSE, ..., na\_matches = "na") Return rows of x that have a match in y. Use to see what will be included in a join.

| A | B | C |
|---|---|---|
| c | v | 3 |

**anti\_join**(x, y, by = NULL, copy = FALSE, ..., na\_matches = "na") Return rows of x that do not have a match in y. Use to see what will not be included in a join.

Use a "**Nest Join**" to inner join one table to another into a nested data frame.

| A | B | C | y              |
|---|---|---|----------------|
| a | t | 1 | <tibble [1x2]> |
| b | u | 2 | <tibble [1x2]> |
| c | v | 3 | <tibble [1x2]> |

**nest\_join**(x, y, by = NULL, copy = FALSE, keep = FALSE, name = NULL, ...) Join data, nesting matches from y in a single new data frame column.

## SET OPERATIONS

| A | B | C |
|---|---|---|
| c | v | 3 |

**intersect(x, y, ...)** Rows that appear in both x and y.

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |

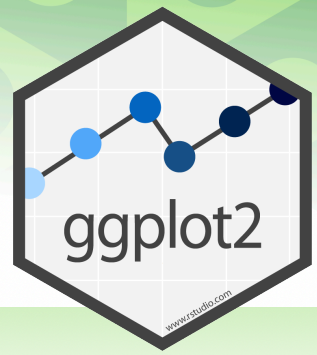
**setdiff(x, y, ...)** Rows that appear in x but not y.

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |
| c | v | 3 |
| d | w | 4 |

**union(x, y, ...)** Rows that appear in x or y, duplicates removed). **union\_all()** retains duplicates.

Use **setequal()** to test whether two data sets contain the exact same rows (in any order).

# Data visualization with ggplot2 : : CHEATSHEET

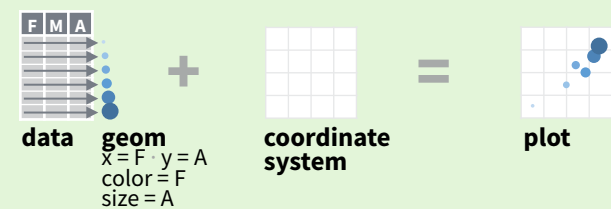


## Basics

**ggplot2** is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

```
ggplot (data = <DATA>) +
<GEOM_FUNCTION> (mapping = aes (<MAPPINGS>),
 stat = <STAT>, position = <POSITION>) +
<COORDINATE_FUNCTION> +
<FACET_FUNCTION> +
<SCALE_FUNCTION> +
<THEME_FUNCTION>
```

**required**

**Not required, sensible defaults supplied**

**ggplot**(data = mpg, aes(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

**last\_plot()** Returns the last plot.

**ggsave**("plot.png", width = 5, height = 5) Saves last plot as 5' x 5' file named "plot.png" in working directory. Matches file type to file extension.

## Aes Common aesthetic values.

**color** and **fill** - string ("red", "#RRGGBB")

**linetype** - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

**size** - integer (in mm for size of points and text)

**linewidth** - integer (in mm for widths of lines)

**shape** - integer/shape name or a single character ("a")

0 1 2 3 4 5 6 7 8 9 10 11 12  
□ ○ △ + × ◇ ▼ ✖ \* ✧ ⊕ ⊗ ⊞ ⊠  
13 14 15 16 17 18 19 20 21 22 23 24 25  
⊞ ⊠ ⊞ ⊠ ⊞ ⊠ ⊞ ⊠ ⊞ ⊠ ⊞ ⊠ ⊞ ⊠



## Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

### GRAPHICAL PRIMITIVES

```
a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))
```

**a + geom\_blank()** and **a + expand\_limits()**  
Ensure limits include values across all plots.

**b + geom\_curve**(aes(yend = lat + 1, xend = long + 1, curvature = 1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size)

**a + geom\_path**(lineend = "butt", linejoin = "round", linemitre = 1) - x, y, alpha, color, group, linetype, size

**a + geom\_polygon**(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

**b + geom\_rect**(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

**a + geom\_ribbon**(aes(ymin = unemploy - 900, ymax = unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

### LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

**b + geom\_abline**(aes(intercept = 0, slope = 1))  
**b + geom\_hline**(aes(yintercept = lat))  
**b + geom\_vline**(aes(xintercept = long))

**b + geom\_segment**(aes(yend = lat + 1, xend = long + 1))  
**b + geom\_spoke**(aes(angle = 1:1155, radius = 1))

### ONE VARIABLE continuous

```
c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)
```

**c + geom\_area**(stat = "bin")  
x, y, alpha, color, fill, linetype, size

**c + geom\_density**(kernel = "gaussian")  
x, y, alpha, color, fill, group, linetype, size, weight

**c + geom\_dotplot()**  
x, y, alpha, color, fill

**c + geom\_freqpoly()**  
x, y, alpha, color, group, linetype, size

**c + geom\_histogram**(binwidth = 5)  
x, y, alpha, color, fill, linetype, size, weight

**c2 + geom\_qq**(aes(sample = hwy))  
x, y, alpha, color, fill, linetype, size, weight

### discrete

```
d <- ggplot(mpg, aes(fl))
```

**d + geom\_bar()**  
x, alpha, color, fill, linetype, size, weight

### TWO VARIABLES

#### both continuous

```
e <- ggplot(mpg, aes(cty, hwy))
```

**e + geom\_label**(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

**e + geom\_point()**  
x, y, alpha, color, fill, shape, size, stroke

**e + geom\_quantile()**  
x, y, alpha, color, group, linetype, size, weight

**e + geom\_rug**(sides = "bl")  
x, y, alpha, color, linetype, size

**e + geom\_smooth**(method = lm)  
x, y, alpha, color, fill, group, linetype, size, weight

**e + geom\_text**(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

#### one discrete, one continuous

```
f <- ggplot(mpg, aes(class, hwy))
```

**f + geom\_col()**  
x, y, alpha, color, fill, group, linetype, size

**f + geom\_boxplot()**  
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

**f + geom\_dotplot**(binaxis = "y", stackdir = "center")  
x, y, alpha, color, fill, group

**f + geom\_violin**(scale = "area")  
x, y, alpha, color, fill, group, linetype, size, weight

#### both discrete

```
g <- ggplot(diamonds, aes(cut, color))
```

**g + geom\_count()**  
x, y, alpha, color, fill, shape, size, stroke

**e + geom\_jitter**(height = 2, width = 2)  
x, y, alpha, color, fill, shape, size

### THREE VARIABLES

```
seals$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))
```

**l + geom\_contour**(aes(z = z))  
x, y, z, alpha, color, group, linetype, size, weight

**l + geom\_contour\_filled**(aes(fill = z))  
x, y, alpha, color, fill, group, linetype, size, subgroup

#### continuous bivariate distribution

```
h <- ggplot(diamonds, aes(carat, price))
```

**h + geom\_bin2d**(binwidth = c(0.25, 500))  
x, y, alpha, color, fill, linetype, size, weight

**h + geom\_density\_2d()**  
x, y, alpha, color, group, linetype, size

**h + geom\_hex()**  
x, y, alpha, color, fill, size

#### continuous function

```
i <- ggplot(economics, aes(date, unemploy))
```

**i + geom\_area()**  
x, y, alpha, color, fill, linetype, size

**i + geom\_line()**  
x, y, alpha, color, group, linetype, size

**i + geom\_step**(direction = "hv")  
x, y, alpha, color, group, linetype, size

#### visualizing error

```
df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))
```

**j + geom\_crossbar**(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

**j + geom\_errorbar()** - x, ymax, ymin, alpha, color, group, linetype, size, width  
Also **geom\_errorbarh()**.

**j + geom\_linerange()**  
x, ymin, ymax, alpha, color, group, linetype, size

**j + geom\_pointrange()** - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

#### maps

Draw the appropriate geometric object depending on the simple features present in the data. **aes()** arguments: **map\_id**, **alpha**, **color**, **fill**, **linetype**, **linewidth**.

```
nc <- sf::st_read(system.file("shape/nc.shp", package = "sf"))
```

**ggplot(nc) + geom\_sf**(aes(fill = AREA))

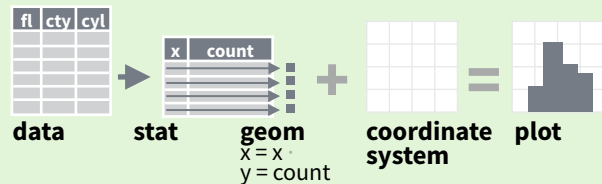
**l + geom\_raster**(aes(fill = z), hjust = 0.5, vjust = 0.5, interpolate = FALSE)  
x, y, alpha, fill

**l + geom\_tile**(aes(fill = z))  
x, y, alpha, color, fill, linetype, size, width

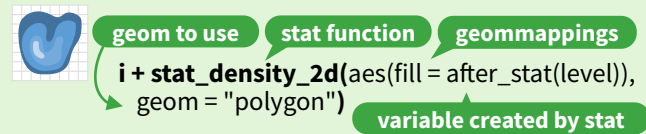
# Stats

An alternative way to build a layer.

A stat builds new variables to plot (e.g., count, prop).



Visualize a stat by changing the default stat of a geom function, **geom\_bar(stat="count")** or by using a stat function, **stat\_count(geom="bar")**, which calls a default geom to make a layer (equivalent to a geom function). Use **after\_stat(name)** syntax to map the stat variable **name** to an aesthetic.



**c + stat\_bin**(binwidth = 1, boundary = 10)  
**x, y** | count, ncount, density, ndensity

**c + stat\_count**(width = 1) **x, y** | count, prop

**c + stat\_density**(adjust = 1, kernel = "gaussian")  
**x, y** | count, density, scaled

**e + stat\_bin\_2d**(bins = 30, drop = T)  
**x, y, fill** | count, density

**e + stat\_bin\_hex**(bins = 30) **x, y, fill** | count, density

**e + stat\_density\_2d**(contour = TRUE, n = 100)  
**x, y, color, size** | level

**e + stat\_ellipse**(level = 0.95, segments = 51, type = "t")

**l + stat\_contour**(aes(z = z)) **x, y, z, order** | level

**l + stat\_summary\_hex**(aes(z = z), bins = 30, fun = max)  
**x, y, z, fill** | value

**l + stat\_summary\_2d**(aes(z = z), bins = 30, fun = mean)  
**x, y, z, fill** | value

**f + stat\_boxplot**(coef = 1.5)  
**x, y** | lower, middle, upper, width, ymin, ymax

**f + stat\_ydensity**(kernel = "gaussian", scale = "area") **x, y** | density, scaled, count, n, violinwidth, width

**e + stat\_ecdf**(n = 40) **x, y** | x, y

**e + stat\_quantile**(quantiles = c(0.1, 0.9), formula = y ~ log(x), method = "rq") **x, y** | quantile

**e + stat\_smooth**(method = "lm", formula = y ~ x, se = T, level = 0.95) **x, y** | se, x, y, ymin, ymax

**ggplot()** + **xlim**(-5, 5) + **stat\_function**(fun = dnorm, n = 20, geom = "point") **x** | x, y

**ggplot()** + **stat\_qq**(aes(sample = 1:100))  
**x, y, sample** | sample, theoretical

**e + stat\_sum**() **x, y, size** | n, prop

**e + stat\_summary**(fun.data = "mean\_cl\_boot")

**h + stat\_summary\_bin**(fun = "mean", geom = "bar")

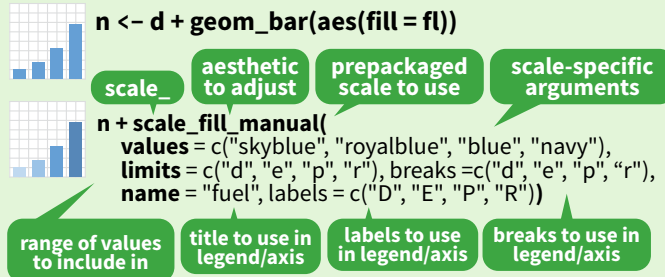
**e + stat\_identity**()

**e + stat\_unique**()

# Scales

Override defaults with **scales** package.

**Scales** map data values to the visual values of an aesthetic. To change a mapping, add a new scale.



## GENERAL PURPOSE SCALES

Use with most aesthetics

**scale\_\*\_continuous()** - Map cont' values to visual ones.

**scale\_\*\_discrete()** - Map discrete values to visual ones.

**scale\_\*\_binned()** - Map continuous values to discrete bins.

**scale\_\*\_identity()** - Use data values as visual ones.

**scale\_\*\_manual**(values = c()) - Map discrete values to manually chosen visual ones.

**scale\_\*\_date**(date\_labels = "%m/%d"), date\_breaks = "2 weeks") - Treat data values as dates.

**scale\_\*\_datetime()** - Treat data values as date times. Same as scale\_\*\_date(). See ?strptime for label formats.

## X & Y LOCATION SCALES

Use with x or y aesthetics (x shown here)

**scale\_x\_log10()** - Plot x on log10 scale.

**scale\_x\_reverse()** - Reverse the direction of the x axis.

**scale\_x\_sqrt()** - Plot x on square root scale.

## COLOR AND FILL SCALES (DISCRETE)

**n + scale\_fill\_brewer**(palette = "Blues")

For palette choices:  
RColorBrewer::display.brewer.all()

**n + scale\_fill\_grey**(start = 0.2, end = 0.8, na.value = "red")

## COLOR AND FILL SCALES (CONTINUOUS)

**o <- c + geom\_dotplot**(aes(fill = x))

**o + scale\_fill\_distiller**(palette = "Blues")

**o + scale\_fill\_gradient**(low="red", high="yellow")

**o + scale\_fill\_gradient2**(low = "red", high = "blue", mid = "white", midpoint = 25)

**o + scale\_fill\_gradientn**(colors = topo.colors(6))  
Also: rainbow(), heat.colors(), terrain.colors(), cm.colors(), RColorBrewer::brewer.pal()

## SHAPE AND SIZE SCALES

**p <- e + geom\_point**(aes(shape = fl, size = cyl))

**p + scale\_shape()** + **scale\_size()**

**p + scale\_shape\_manual**(values = c(3:7))

**0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25**

**p + scale\_radius**(range = c(1,6))

**p + scale\_size\_area**(max\_size = 6)

# Coordinate Systems

**r <- d + geom\_bar()**

**r + coord\_cartesian**(xlim = c(0, 5)) - xlim, ylim  
The default cartesian coordinate system.

**r + coord\_fixed**(ratio = 1/2)  
ratio, xlim, ylim - Cartesian coordinates with fixed aspect ratio between x and y units.

**r + coord\_flip()**  
Flip cartesian coordinates by switching x and y aesthetic mappings.

**r + coord\_polar**(theta = "x", direction=1)  
theta, start, direction - Polar coordinates.

**r + coord\_trans**(y = "sqrt") - x, y, xlim, ylim  
Transformed cartesian coordinates. Set xtrans and ytrans to the name of a window function.

**π + coord\_sf**() - xlim, ylim, crs. Ensures all layers use a common Coordinate Reference System.

# Position Adjustments

Position adjustments determine how to arrange geoms that would otherwise occupy the same space.

**s <- ggplot**(mpg, aes(fl, fill = drv))

**s + geom\_bar**(position = "dodge")  
Arrange elements side by side.

**s + geom\_bar**(position = "fill")  
Stack elements on top of one another, normalize height.

**e + geom\_point**(position = "jitter")  
Add random noise to X and Y position of each element to avoid overplotting.

**e + geom\_label**(position = "nudge")  
Nudge labels away from points.

**s + geom\_bar**(position = "stack")  
Stack elements on top of one another.

Each position adjustment can be recast as a function with manual **width** and **height** arguments:

**s + geom\_bar**(position = position\_dodge(width = 1))

# Themes

**r + theme\_bw**()  
White background with grid lines.

**r + theme\_gray**()  
Grey background (default theme).

**r + theme\_dark**()  
Dark for contrast.

**r + theme\_classic**()  
**r + theme\_light**()  
**r + theme\_linedraw**()  
**r + theme\_minimal**()  
**r + theme\_void**()

**r + theme**()  
Customize aspects of the theme such as axis, legend, panel, and facet properties.

**r + labs**(title = "Title") + **theme**(plot.title.position = "plot")

**r + theme**(panel.background = element\_rect(fill = "blue"))

# Faceting

Facets divide a plot into subplots based on the values of one or more discrete variables.

**t <- ggplot**(mpg, aes(cty, hwy)) + **geom\_point**()

**t + facet\_grid**(. ~ fl)  
Facet into columns based on fl.

**t + facet\_grid**(year ~ .)  
Facet into rows based on year.

**t + facet\_grid**(year ~ fl)  
Facet into both rows and columns.

**t + facet\_wrap**(~ fl)  
Wrap facets into a rectangular layout.

Set **scales** to let axis limits vary across facets.

**t + facet\_grid**(drv ~ fl, scales = "free")  
x and y axis limits adjust to individual facets:  
"free\_x" - x axis limits adjust  
"free\_y" - y axis limits adjust

Set **labeller** to adjust facet label:

**t + facet\_grid**(. ~ fl, labeller = label\_both)

**fl: c** **fl: d** **fl: e** **fl: p** **fl: r**

**t + facet\_grid**(fl ~ ., labeller = label\_bquote(alpha ^ .(fl)))

$\alpha^c$   $\alpha^d$   $\alpha^e$   $\alpha^p$   $\alpha^r$

# Labels and Legends

Use **labs()** to label the elements of your plot.

**t + labs**(x = "New x axis label", y = "New y axis label", title = "Add a title above the plot", subtitle = "Add a subtitle below title", caption = "Add a caption below plot", alt = "Add alt text to the plot", <AES> = "New <AES> legend title")

**t + annotate**(geom = "text", x = 8, y = 9, label = "A")  
Places a geom with manually selected aesthetics.

**p + guides**(x = guide\_axis(n.dodge = 2))  
Avoid crowded or overlapping labels with guide\_axis(n.dodge or angle).

**n + guides**(fill = "none")  
Set legend type for each aesthetic: colorbar, legend, or none (no legend).

**n + theme**(legend.position = "bottom")  
Place legend at "bottom", "top", "left", or "right".

**n + scale\_fill\_discrete**(name = "Title", labels = c("A", "B", "C", "D", "E"))  
Set legend title and labels with a scale function.

# Zooming

**Without clipping** (preferred):

**t + coord\_cartesian**(xlim = c(0, 100), ylim = c(10, 20))

**With clipping** (removes unseen data points):

**t + xlim**(0, 100) + **ylim**(10, 20)

**t + scale\_x\_continuous**(limits = c(0, 100)) + **scale\_y\_continuous**(limits = c(0, 100))

# Data import with the tidyverse : : CHEATSHEET



## Read Tabular Data with readr

**read\_\***(file, col\_names = TRUE, col\_types = NULL, col\_select = NULL, id = NULL, locale, n\_max = Inf, skip = 0, na = c("", "NA"), guess\_max = min(1000, n\_max), show\_col\_types = TRUE) See ?**read\_delim**

A|B|C  
1|2|3  
4|5|NA



| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

**read\_delim**("file.txt", delim = "|") Read files with any delimiter. If no delimiter is specified, it will automatically guess.  
To make file.txt, run: write\_file("A|B|C\n1|2|3\n4|5|NA", file = "file.txt")

A,B,C  
1,2,3  
4,5,NA



| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

**read\_csv**("file.csv") Read a comma delimited file with period decimal marks.  
write\_file("A,B,C\n1,2,3\n4,5,NA", file = "file.csv")

A;B;C  
1,5;2;3  
4,5;5;NA



| A   | B | C  |
|-----|---|----|
| 1.5 | 2 | 3  |
| 4.5 | 5 | NA |

**read\_csv2**("file2.csv") Read semicolon delimited files with comma decimal marks.  
write\_file("A;B;C\n1,5;2;3\n4,5;5;NA", file = "file2.csv")

A B C  
1 2 3  
4 5 NA



| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

**read\_tsv**("file.tsv") Read a tab delimited file. Also **read\_table()**.  
**read\_fwf**("file.tsv", fwf\_widths(c(2, 2, NA))) Read a fixed width file.  
write\_file("A\tB\tC\n1\t2\t3\n4\t5\tNA\n", file = "file.tsv")

### USEFUL READ ARGUMENTS

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |

#### No header

read\_csv("file.csv", col\_names = FALSE)

| 1 | 2 | 3  |
|---|---|----|
| 4 | 5 | NA |

#### Skip lines

read\_csv("file.csv", skip = 1)

| x | y | z  |
|---|---|----|
| A | B | C  |
| 1 | 2 | 3  |
| 4 | 5 | NA |

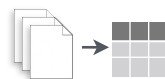
#### Provide header

read\_csv("file.csv",  
col\_names = c("x", "y", "z"))

| A | B | C |
|---|---|---|
| 1 | 2 | 3 |

#### Read a subset of lines

read\_csv("file.csv", n\_max = 1)



#### Read multiple files into a single table

read\_csv(c("f1.csv", "f2.csv", "f3.csv"),  
id = "origin\_file")

| A  | B | C  |
|----|---|----|
| NA | 2 | 3  |
| 4  | 5 | NA |

#### Read values as missing

read\_csv("file.csv", na = c("1"))

A;B;C  
1,5;2;3,0

#### Specify decimal marks

read\_delim("file2.csv", locale =  
locale(decimal\_mark = ";"))

## Save Data with readr

**write\_\***(x, file, na = "NA", append, col\_names, quote, escape, eol, num\_threads, progress)

| A | B | C  |
|---|---|----|
| 1 | 2 | 3  |
| 4 | 5 | NA |



A,B,C  
1,2,3  
4,5,NA

**write\_delim**(x, file, delim = " ") Write files with any delimiter.

**write\_csv**(x, file) Write a comma delimited file.

**write\_csv2**(x, file) Write a semicolon delimited file.

**write\_tsv**(x, file) Write a tab delimited file.

One of the first steps of a project is to import outside data into R. Data is often stored in tabular formats, like csv files or spreadsheets.



The front page of this sheet shows how to import and save text files into R using **readr**.



The back page shows how to import spreadsheet data from Excel files using **readxl** or Google Sheets using **googlesheets4**.

## Column Specification with readr

Column specifications define what data type each column of a file will be imported as. By default readr will generate a column spec when a file is read and output a summary.

**spec(x)** Extract the full column specification for the given imported data frame.

```
spec(x)
cols(
age = col_integer(),
edu = col_character(),
earn = col_double()
)
```

age is an integer

earn is a double (numeric)

edu is a character

### COLUMN TYPES

Each column type has a function and corresponding string abbreviation.

- col\_logical()** - "l"
- col\_integer()** - "i"
- col\_double()** - "d"
- col\_number()** - "n"
- col\_character()** - "c"
- col\_factor(levels, ordered = FALSE)** - "f"
- col\_datetime(format = "")** - "T"
- col\_date(format = "")** - "D"
- col\_time(format = "")** - "t"
- col\_skip()** - "-", "\_"
- col\_guess()** - "?"

### OTHER TYPES OF DATA

Try one of the following packages to import other types of files:

- haven** - SPSS, Stata, and SAS files
- DBI** - databases
- jsonlite** - json
- xml2** - XML
- httr** - Web APIs
- rvest** - HTML (Web Scraping)
- readr::read\_lines()** - text data

### USEFUL COLUMN ARGUMENTS

#### Hide col spec message

read\_\*(file, show\_col\_types = FALSE)

#### Select columns to import

Use names, position, or selection helpers.

read\_\*(file, col\_select = c(age, earn))

#### Guess column types

To guess a column type, read\_\*(file) looks at the first 1000 rows of data. Increase with **guess\_max**.  
read\_\*(file, guess\_max = Inf)

### DEFINE COLUMN SPECIFICATION

#### Set a default type

```
read_csv(
 file,
 col_type = list(default = col_double())
)
```

#### Use column type or string abbreviation

```
read_csv(
 file,
 col_type = list(x = col_double(), y = "l", z = "_")
)
```

#### Use a single string of abbreviations

```
col types: skip, guess, integer, logical, character
read_csv(
 file,
 col_type = "_?ilc"
)
```

# Import Spreadsheets with readxl

## READ EXCEL FILES

|   | A  | B  | C  | D  | E  |
|---|----|----|----|----|----|
| 1 | x1 | x2 | x3 | x4 | x5 |
| 2 | x  |    | z  | 8  |    |
| 3 | y  | 7  |    | 9  | 10 |

s1

**read\_excel(path, sheet = NULL, range = NULL)**  
Read a .xls or .xlsx file based on the file extension. See front page for more read arguments. Also **read\_xls()** and **read\_xlsx()**.  
`read_excel("excel_file.xlsx")`

## READ SHEETS

| A | B | C | D | E |
|---|---|---|---|---|
|   |   |   |   |   |
|   |   |   |   |   |

s1 s2 s3

**read\_excel(path, sheet = NULL)** Specify which sheet to read by position or name.  
`read_excel(path, sheet = 1)`  
`read_excel(path, sheet = "s1")`

|    |    |    |
|----|----|----|
| s1 | s2 | s3 |
|----|----|----|

**excel\_sheets(path)** Get a vector of sheet names.  
`excel_sheets("excel_file.xlsx")`

| A | B | C | D | E |
|---|---|---|---|---|
|   |   |   |   |   |
|   |   |   |   |   |

s1

| A | B | C | D | E |
|---|---|---|---|---|
|   |   |   |   |   |
|   |   |   |   |   |

s1 s2 s3

**To read multiple sheets:**

1. Get a vector of sheet names from the file path.
2. Set the vector names to be the sheet names.
3. Use `purrr::map()` and `purrr::list_rbind()` to read multiple files into one

`path <- "your_file_path.xlsx"` data frame.

```
path |>
 excel_sheets() |>
 set_names() |>
 map(read_excel, path = path) |>
 list_rbind()
```

## OTHER USEFUL EXCEL PACKAGES

For functions to write data to Excel files, see:

- **openxlsx**
- **writexl**

For working with non-tabular Excel data, see:

- **tidyxl**



## READXL COLUMN SPECIFICATION

Column specifications define what data type each column of a file will be imported as.

Use the **col\_types** argument of **read\_excel()** to set the column specification.

### Guess column types

To guess a column type, `read_excel()` looks at the first 1000 rows of data. Increase with the **guess\_max** argument.

`read_excel(path, guess_max = Inf)`

### Set all columns to same type, e.g. character

`read_excel(path, col_types = "text")`

### Set each column individually

```
read_excel(
 path,
 col_types = c("text", "guess", "guess", "numeric")
)
```

## COLUMN TYPES

| logical | numeric | text  | date       | list  |
|---------|---------|-------|------------|-------|
| TRUE    | 2       | hello | 1947-01-08 | hello |
| FALSE   | 3.45    | world | 1956-10-21 | 1     |

- skip
- guess
- logical
- numeric
- text
- date
- list

Use **list** for columns that include multiple data types. See **tidyr** and **purrr** for list-column data.

| A | B | C | D | E |
|---|---|---|---|---|
| 1 | 1 | 2 | 3 | 4 |
| 2 | x |   | y | z |
| 3 | 6 | 7 |   | 9 |

s1

|    |   |   |
|----|---|---|
| 2  | 3 | 4 |
| NA | y | z |

## CELL SPECIFICATION FOR READXL AND GOOGLESHEETS4

Use the **range** argument of **readxl::read\_excel()** or **googlesheets4::read\_sheet()** to read a subset of cells from a sheet.

```
read_excel(path, range = "Sheet1!B1:D2")
read_sheet(ss, range = "B1:D2")
```

Also use the range argument with cell specification functions **cell\_limits()**, **cell\_rows()**, **cell\_cols()**, and **anchored()**.

# with googlesheets4

## READ SHEETS

|   | A  | B  | C  | D  | E  |
|---|----|----|----|----|----|
| 1 | x1 | x2 | x3 | x4 | x5 |
| 2 | x  |    | z  | 8  |    |
| 3 | y  | 7  |    | 9  | 10 |

s1

**read\_sheet(ss, sheet = NULL, range = NULL)**  
Read a sheet from a URL, a Sheet ID, or a dribble from the googledrive package. See front page for more read arguments. Same as **range\_read()**.

## SHEETS METADATA

**URLs** are in the form:

`https://docs.google.com/spreadsheets/d/  
SPREADSHEET_ID/edit#gid=`**SHEET\_ID**

**gs4\_get(ss)** Get spreadsheet meta data.

**gs4\_find(...)** Get data on all spreadsheet files.

**sheet\_properties(ss)** Get a tibble of properties for each worksheet. Also **sheet\_names()**.

## WRITE SHEETS

|   |   |   |
|---|---|---|
| 1 | x | 4 |
| 2 | y | 5 |
| 3 | z | 6 |

s1

| A | B | C | D |
|---|---|---|---|
| 1 |   |   |   |
| 2 |   |   |   |

s1

| x1 | x2 | x3 |
|----|----|----|
| 2  | y  | 5  |
| 3  | z  | 6  |

s1

**write\_sheet(data, ss = NULL, sheet = NULL)**  
Write a data frame into a new or existing Sheet.

**gs4\_create(name, ..., sheets = NULL)** Create a new Sheet with a vector of names, a data frame, or a (named) list of data frames.

**sheet\_append(ss, data, sheet = 1)** Add rows to the end of a worksheet.



## GOOGLESHEETS4 COLUMN SPECIFICATION

Column specifications define what data type each column of a file will be imported as.

Use the **col\_types** argument of **read\_sheet()**/**range\_read()** to set the column specification.

### Guess column types

To guess a column type `read_sheet()`/`range_read()` looks at the first 1000 rows of data. Increase with **guess\_max**.

`read_sheet(path, guess_max = Inf)`

### Set all columns to same type, e.g. character

`read_sheet(path, col_types = "c")`

### Set each column individually

# col types: skip, guess, integer, logical, character  
`read_sheets(ss, col_types = "_?ilc")`

## COLUMN TYPES

| l     | n    | c     | D          | L     |
|-------|------|-------|------------|-------|
| TRUE  | 2    | hello | 1947-01-08 | hello |
| FALSE | 3.45 | world | 1956-10-21 | 1     |

- skip - "\_" or "-"
- guess - "?"
- logical - "l"
- integer - "i"
- double - "d"
- numeric - "n"
- date - "D"
- datetime - "T"
- character - "c"
- list-column - "L"
- cell - "C" Returns list of raw cell data.

Use list for columns that include multiple data types. See **tidyr** and **purrr** for list-column data.

## FILE LEVEL OPERATIONS

**googlesheets4** also offers ways to modify other aspects of Sheets (e.g. freeze rows, set column width, manage (work)sheets). Go to **googlesheets4.tidyverse.org** to read more.

For whole-file operations (e.g. renaming, sharing, placing within a folder), see the tidyverse package **googledrive** at **googledrive.tidyverse.org**.

# String manipulation with stringr : : CHEATSHEET

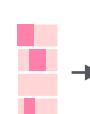


The **stringr** package provides a set of internally consistent tools for working with character strings, i.e. sequences of characters surrounded by quotation marks.

## Detect Matches

 **str\_detect**(string, **pattern**, negate = FALSE)  
Detect the presence of a pattern match in a string. Also **str\_like()**. `str_detect(fruit, "a")`

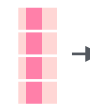
 **str\_starts**(string, **pattern**, negate = FALSE)  
Detect the presence of a pattern match at the beginning of a string. Also **str\_ends()**. `str_starts(fruit, "a")`

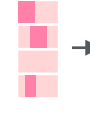
 **str\_which**(string, **pattern**, negate = FALSE)  
Find the indexes of strings that contain a pattern match. `str_which(fruit, "a")`

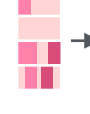
 **str\_locate**(string, **pattern**) Locate the positions of pattern matches in a string. Also **str\_locate\_all()**. `str_locate(fruit, "a")`

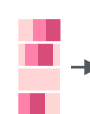
 **str\_count**(string, **pattern**) Count the number of matches in a string. `str_count(fruit, "a")`

## Subset Strings

 **str\_sub**(string, start = 1L, end = -1L) Extract substrings from a character vector. `str_sub(fruit, 1, 3); str_sub(fruit, -2)`

 **str\_subset**(string, **pattern**, negate = FALSE)  
Return only the strings that contain a pattern match. `str_subset(fruit, "p")`

 **str\_extract**(string, **pattern**) Return the first pattern match found in each string, as a vector. Also **str\_extract\_all()** to return every pattern match. `str_extract(fruit, "[aeiou]")`

 **str\_match**(string, **pattern**) Return the first pattern match found in each string, as a matrix with a column for each ( ) group in pattern. Also **str\_match\_all()**. `str_match(sentences, "(a|the) ([^+ ])")`

## Manage Lengths

 **str\_length**(string) The width of strings (i.e. number of code points, which generally equals the number of characters). `str_length(fruit)`

 **str\_pad**(string, width, side = c("left", "right", "both"), pad = " ") Pad strings to constant width. `str_pad(fruit, 17)`

 **str\_trunc**(string, width, side = c("right", "left", "center"), ellipsis = "...") Truncate the width of strings, replacing content with ellipsis. `str_trunc(sentences, 6)`

 **str\_trim**(string, side = c("both", "left", "right")) Trim whitespace from the start and/or end of a string. `str_trim(str_pad(fruit, 17))`

**str\_squish**(string) Trim whitespace from each end and collapse multiple spaces into single spaces. `str_squish(str_pad(fruit, 17, "both"))`

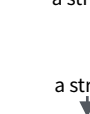
## Mutate Strings

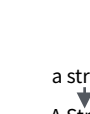
 **str\_sub()** <- value. Replace substrings by identifying the substrings with **str\_sub()** and assigning into the results. `str_sub(fruit, 1, 3) <- "str"`

 **str\_replace**(string, **pattern**, replacement)  
Replace the first matched pattern in each string. Also **str\_remove()**. `str_replace(fruit, "p", "-")`

 **str\_replace\_all**(string, **pattern**, replacement)  
Replace all matched patterns in each string. Also **str\_remove\_all()**. `str_replace_all(fruit, "p", "-")`

 **str\_to\_lower**(string, locale = "en")<sup>1</sup>  
Convert strings to lower case. `str_to_lower(sentences)`

 **str\_to\_upper**(string, locale = "en")<sup>1</sup>  
Convert strings to upper case. `str_to_upper(sentences)`

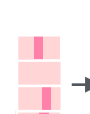
 **str\_to\_title**(string, locale = "en")<sup>1</sup> Convert strings to title case. Also **str\_to\_sentence()**. `str_to_title(sentences)`

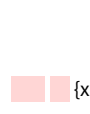
## Join and Split

 **str\_c**(..., sep = "", collapse = NULL) Join multiple strings into a single string. `str_c(letters, LETTERS)`

 **str\_flatten**(string, collapse = "") Combines into a single string, separated by collapse. `str_flatten(fruit, ",")`

 **str\_dup**(string, times) Repeat strings times times. Also **str\_unique()** to remove duplicates. `str_dup(fruit, times = 2)`

 **str\_split\_fixed**(string, **pattern**, n) Split a vector of strings into a matrix of substrings (splitting at occurrences of a pattern match). Also **str\_split()** to return a list of substrings and **str\_split\_i()** to return the ith substring. `str_split_fixed(sentences, " ", n=3)`

 **str\_glue**(..., .sep = "", .envir = parent.frame()) Create a string from strings and {expressions} to evaluate. `str_glue("Pi is {pi}")`

 **str\_glue\_data**(.x, ..., .sep = "", .envir = parent.frame(), .na = "NA") Use a data frame, list, or environment to create a string from strings and {expressions} to evaluate. `str_glue_data(mtcars, "{rownames(mtcars)} has {hp} hp")`

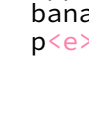
## Order Strings

 **str\_order**(x, decreasing = FALSE, na\_last = TRUE, locale = "en", numeric = FALSE, ...) <sup>1</sup>  
Return the vector of indexes that sorts a character vector. `fruit[str_order(fruit)]`

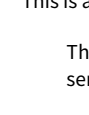
 **str\_sort**(x, decreasing = FALSE, na\_last = TRUE, locale = "en", numeric = FALSE, ...) <sup>1</sup>  
Sort a character vector. `str_sort(fruit)`

## Helpers

 **str\_conv**(string, encoding) Override the encoding of a string. `str_conv(fruit, "ISO-8859-1")`

 **str\_view**(string, **pattern**, match = NA)  
View HTML rendering of all regex matches. `str_view(sentences, "[aeiou]")`

 **str\_equal**(x, y, locale = "en", ignore\_case = FALSE, ...) <sup>1</sup> Determine if two strings are equivalent. `str_equal(c("a", "b"), c("a", "c"))`

 **str\_wrap**(string, width = 80, indent = 0, exdent = 0) Wrap strings into nicely formatted paragraphs. `str_wrap(sentences, 20)`

<sup>1</sup> See [bit.ly/ISO639-1](https://bit.ly/ISO639-1) for a complete list of locales.

# Need to Know

Pattern arguments in stringr are interpreted as regular expressions *after any special characters have been parsed*.

In R, you write regular expressions as *strings*, sequences of characters surrounded by quotes ("" or '') or single quotes('').

Some characters cannot be represented directly in an R string. These must be represented as **special characters**, sequences of characters that have a specific meaning., e.g.

| Special Character | Represents |
|-------------------|------------|
| \\                | \          |
| \"                | "          |
| \n                | new line   |

Run `?\"` to see a complete list

Because of this, whenever a \ appears in a regular expression, you must write it as \\ in the string that represents the regular expression.

Use `writeLines()` to see how R views your string after all special characters have been parsed.

```
writeLines("\\.")
\
```

```
writeLines("\\ is a backslash")
\ is a backslash
```

## INTERPRETATION

Patterns in stringr are interpreted as regexs. To change this default, wrap the pattern in one of:

**regex()** (pattern, ignore\_case = FALSE, multiline = FALSE, comments = FALSE, dotall = FALSE, ...) Modifies a regex to ignore cases, match end of lines as well of end of strings, allow R comments within regex's, and/or to have . match everything including \n.  
`str_detect("i", regex("i", TRUE))`

**fixed()** Matches raw bytes but will miss some characters that can be represented in multiple ways (fast). `str_detect("\u0130", fixed("i"))`

**coll()** Matches raw bytes and will use locale specific collation rules to recognize characters that can be represented in multiple ways (slow). `str_detect("\u0130", coll("i", TRUE, locale = "tr"))`

**boundary()** Matches boundaries between characters, line\_breaks, sentences, or words. `str_split(sentences, boundary("word"))`

# Regular Expressions - Regular expressions, or *regexps*, are a concise language for describing patterns in strings.

## MATCH CHARACTERS

| string<br>(type this) | regex<br>(to mean this)      | matches<br>(which matches this)                           | example                                          |
|-----------------------|------------------------------|-----------------------------------------------------------|--------------------------------------------------|
|                       | <b>a (etc.)</b>              | a (etc.)                                                  | <code>see("a")</code> abc ABC 123 .!?\()\        |
| \\.                   | \\.                          | .                                                         | <code>see("\\.")</code> abc ABC 123 .!?\()\      |
| \\!                   | \\!                          | !                                                         | <code>see("\\!")</code> abc ABC 123 .!?\()\      |
| \\?                   | \\?                          | ?                                                         | <code>see("\\?")</code> abc ABC 123 .!?\()\      |
| \\\\                  | \\\\                         | \\                                                        | <code>see("\\\\")</code> abc ABC 123 .!?\()\     |
| \\(                   | \\(                          | (                                                         | <code>see("\\(")</code> abc ABC 123 .!?\()\      |
| \\)                   | \\)                          | )                                                         | <code>see("\\)")</code> abc ABC 123 .!?\()\      |
| \\{                   | \\{                          | {                                                         | <code>see("\\{")</code> abc ABC 123 .!?\()\      |
| \\}                   | \\}                          | }                                                         | <code>see("\\}")</code> abc ABC 123 .!?\()\      |
| \\n                   | \\n                          | new line (return)                                         | <code>see("\\n")</code> abc ABC 123 .!?\()\      |
| \\t                   | \\t                          | tab                                                       | <code>see("\\t")</code> abc ABC 123 .!?\()\      |
| \\s                   | \\s                          | any whitespace ( <b>S</b> for <i>non-whitespaces</i> )    | <code>see("\\s")</code> abc ABC 123 .!?\()\      |
| \\d                   | \\d                          | any digit ( <b>D</b> for <i>non-digits</i> )              | <code>see("\\d")</code> abc ABC 123 .!?\()\      |
| \\w                   | \\w                          | any word character ( <b>W</b> for <i>non-word chars</i> ) | <code>see("\\w")</code> abc ABC 123 .!?\()\      |
| \\b                   | \\b                          | word boundaries                                           | <code>see("\\b")</code> abc ABC 123 .!?\()\      |
|                       | <b>[digit:]</b> <sup>1</sup> | digits                                                    | <code>see("[digit:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[alpha:]</b> <sup>1</sup> | letters                                                   | <code>see("[alpha:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[lower:]</b> <sup>1</sup> | lowercase letters                                         | <code>see("[lower:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[upper:]</b> <sup>1</sup> | uppercase letters                                         | <code>see("[upper:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[alnum:]</b> <sup>1</sup> | letters and numbers                                       | <code>see("[alnum:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[punct:]</b> <sup>1</sup> | punctuation                                               | <code>see("[punct:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[graph:]</b> <sup>1</sup> | letters, numbers, and punctuation                         | <code>see("[graph:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[space:]</b> <sup>1</sup> | space characters (i.e. \\s)                               | <code>see("[space:]")</code> abc ABC 123 .!?\()\ |
|                       | <b>[blank:]</b> <sup>1</sup> | space and tab (but not new line)                          | <code>see("[blank:]")</code> abc ABC 123 .!?\()\ |
|                       | .                            | every character except a new line                         | <code>see(".")</code> abc ABC 123 .!?\()\        |

<sup>1</sup> Many base R functions require classes to be wrapped in a second set of [ ], e.g. `[digit:]`

|                 |
|-----------------|
| <b>[space:]</b> |
| ↵ new line      |
| <b>[blank:]</b> |
| □ space         |
| □ tab           |



|                     |                  |
|---------------------|------------------|
| <b>[graph:]</b>     |                  |
| <b>[punct:]</b>     | <b>[symbol:]</b> |
| . , : ; ? ! / * @ # | ` = + ^          |
| - _ " ' [ ] { } ( ) | ~ < > \$         |
| <b>[alnum:]</b>     |                  |
| <b>[digit:]</b>     |                  |
| 0 1 2 3 4 5 6 7 8 9 |                  |
| <b>[alpha:]</b>     |                  |
| <b>[lower:]</b>     | <b>[upper:]</b>  |
| a b c d e f         | A B C D E F      |
| g h i j k l         | G H I J K L      |
| m n o p q r         | M N O P Q R      |
| s t u v w x         | S T U V W X      |
| y z                 | Y Z              |

## ALTERNATES

`alt <- function(rx) str_view("abcde", rx)`

| regex        | matches      | example                         |
|--------------|--------------|---------------------------------|
| <b>ab d</b>  | or           | <code>alt("ab d")</code> abcde  |
| <b>[abe]</b> | one of       | <code>alt("[abe]")</code> abcde |
| <b>^abe</b>  | anything but | <code>alt("^abe")</code> abcde  |
| <b>[a-c]</b> | range        | <code>alt("[a-c]")</code> abcde |

## ANCHORS

`anchor <- function(rx) str_view("aaa", rx)`

| regex      | matches         | example                        |
|------------|-----------------|--------------------------------|
| <b>^a</b>  | start of string | <code>anchor("^a")</code> aaa  |
| <b>a\$</b> | end of string   | <code>anchor("a\$")</code> aaa |

## LOOK AROUNDS

`look <- function(rx) str_view("bacad", rx)`

| regex             | matches         | example                               |
|-------------------|-----------------|---------------------------------------|
| <b>a(=?c)</b>     | followed by     | <code>look("a(=?c)")</code> bacad     |
| <b>a(!c)</b>      | not followed by | <code>look("a(!c)")</code> bacad      |
| <b>(?&lt;=b)a</b> | preceded by     | <code>look("(?&lt;=b)a")</code> bacad |
| <b>(?&lt;!b)a</b> | not preceded by | <code>look("(?&lt;!b)a")</code> bacad |

## QUANTIFIERS

`quant <- function(rx) str_view(".a.aa.aaa", rx)`

| regex         | matches         | example                                |
|---------------|-----------------|----------------------------------------|
| <b>a?</b>     | zero or one     | <code>quant("a?")</code> .a.aa.aaa     |
| <b>a*</b>     | zero or more    | <code>quant("a*")</code> .a.aa.aaa     |
| <b>a+</b>     | one or more     | <code>quant("a+")</code> .a.aa.aaa     |
| <b>a{n}</b>   | exactly n       | <code>quant("a{2}")</code> .a.aa.aaa   |
| <b>a{n,}</b>  | n or more       | <code>quant("a{2,}")</code> .a.aa.aaa  |
| <b>a{n,m}</b> | between n and m | <code>quant("a{2,4}")</code> .a.aa.aaa |

## GROUPS

`ref <- function(rx) str_view("abbaab", rx)`

Use parentheses to set precedent (order of evaluation) and create groups

| regex          | matches         | example                           |
|----------------|-----------------|-----------------------------------|
| <b>(ab d)e</b> | sets precedence | <code>alt("(ab d)e")</code> abcde |

Use an escaped number to refer to and duplicate parentheses groups that occur earlier in a pattern. Refer to each group by its order of appearance

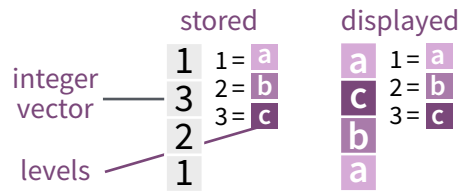
| string<br>(type this) | regex<br>(to mean this) | matches<br>(which matches this) | example<br>(the result is the same as ref("abba")) |
|-----------------------|-------------------------|---------------------------------|----------------------------------------------------|
| <b>\\1</b>            | <b>\\1</b> (etc.)       | first () group, etc.            | <code>ref("(a)(b)\\2\\1")</code> abbaab            |

# Factors with forcats : : CHEATSHEET

The **forcats** package provides tools for working with factors, which are R's data structure for categorical data.

## Factors

R represents categorical data with factors. A **factor** is an integer vector with a **levels** attribute that stores a set of mappings between integers and categorical values. When you view a factor, R displays not the integers, but the levels associated with them.



Create a factor with `factor()`

**factor**(x = character(), levels, labels = levels, exclude = NA, ordered = is.ordered(x), nmax = NA) Convert a vector to a factor. Also **as\_factor()**.  
`f <- factor(c("a", "c", "b", "a"), levels = c("a", "b", "c"))`

Return its levels with `levels()`  
**levels**(x) Return/set the levels of a factor. `levels(f)`; `levels(f) <- c("x", "y", "z")`

Use `unclass()` to see its structure

## Inspect Factors

**fct\_count**(f, sort = FALSE, prop = FALSE) Count the number of values with each level. `fct_count(f)`

**fct\_match**(f, lvls) Check for lvls in f. `fct_match(f, "a")`

**fct\_unique**(f) Return the unique values, removing duplicates. `fct_unique(f)`

## Combine Factors

**fct\_c**(...) Combine factors with different levels. Also **fct\_cross**().  
`f1 <- factor(c("a", "c"))`  
`f2 <- factor(c("b", "a"))`  
`fct_c(f1, f2)`

**fct\_unify**(fs, levels = lvls\_union(fs)) Standardize levels across a list of factors.  
`fct_unify(list(f2, f1))`

## Change the order of levels

**fct\_relevel**(f, ..., after = 0L) Manually reorder factor levels.  
`fct_relevel(f, c("b", "c", "a"))`

**fct\_infreq**(f, ordered = NA) Reorder levels by the frequency in which they appear in the data (highest frequency first). Also **fct\_inseq()**.  
`f3 <- factor(c("c", "c", "a"))`  
`fct_infreq(f3)`

**fct\_inorder**(f, ordered = NA) Reorder levels by order in which they appear in the data.  
`fct_inorder(f2)`

**fct\_rev**(f) Reverse level order.  
`f4 <- factor(c("a", "b", "c"))`  
`fct_rev(f4)`

**fct\_shift**(f) Shift levels to left or right, wrapping around end.  
`fct_shift(f4)`

**fct\_shuffle**(f, n = 1L) Randomly permute order of factor levels.  
`fct_shuffle(f4)`

**fct\_reorder**(f, x, fun = median, ..., desc = FALSE) Reorder levels by their relationship with another variable.  
`boxplot(PlantGrowth, weight ~ fct_reorder(group, weight))`

**fct\_reorder2**(f, x, y, fun = last2, ..., desc = TRUE) Reorder levels by their final values when plotted with two other variables.  
`ggplot(diamonds, aes(carat, price, color = fct_reorder2(color, carat, price))) + geom_smooth()`

## Change the value of levels

**fct\_recode**(f, ...) Manually change levels. Also **fct\_relabel**() which obeys `purrr::map` syntax to apply a function or expression to each level.  
`fct_recode(f, v = "a", x = "b", z = "c")`  
`fct_relabel(f, ~ paste0("x", .x))`

**fct\_anon**(f, prefix = "") Anonymize levels with random integers.  
`fct_anon(f)`

**fct\_collapse**(f, ..., other\_level = NULL) Collapse levels into manually defined groups.  
`fct_collapse(f, x = c("a", "b"))`

**fct\_lump\_min**(f, min, w = NULL, other\_level = "Other") Lumps together factors that appear fewer than min times. Also **fct\_lump\_n()**, **fct\_lump\_prop()**, and **fct\_lump\_lowfreq()**.  
`fct_lump_min(f, min = 2)`

**fct\_other**(f, keep, drop, other\_level = "Other") Replace levels with "other".  
`fct_other(f, keep = c("a", "b"))`

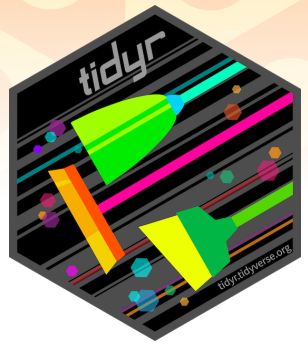
## Add or drop levels

**fct\_drop**(f, only) Drop unused levels.  
`f5 <- factor(c("a", "b"), c("a", "b", "x"))`  
`f6 <- fct_drop(f5)`

**fct\_expand**(f, ...) Add levels to a factor.  
`fct_expand(f6, "x")`

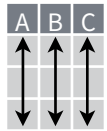
**fct\_na\_value\_to\_level**(f, level = "(Missing)") Assigns a level to NAs to ensure they appear in plots, etc.  
`f7 <- factor(c("a", "b", NA))`  
`fct_na_value_to_level(f7, level = "(Missing)")`

# Data tidying with tidyr :: CHEATSHEET



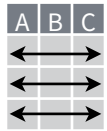
**Tidy data** is a way to organize tabular data in a consistent data structure across packages.

A table is tidy if:



Each **variable** is in its own **column**

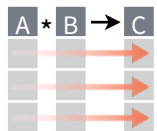
&



Each **observation**, or **case**, is in its own row



Access **variables** as **vectors**



Preserve **cases** in vectorized operations

## Tibbles

### AN ENHANCED DATA FRAME

Tibbles are a table format provided by the **tibble** package. They inherit the data frame class, but have improved behaviors:

- **Subset** a new tibble with `[],` a vector with `[[` and `$.`
- **No partial matching** when subsetting columns.
- **Display** concise views of the data on one screen.

**options(tibble.print\_max = n, tibble.print\_min = m, tibble.width = Inf)** Control default display settings.

**View()** or **glimpse()** View the entire data set.

### CONSTRUCT A TIBBLE

**tibble(...)** Construct by columns.

`tibble(x = 1:3, y = c("a", "b", "c"))`

**tribble(...)** Construct by rows.

```
tribble(~x, ~y,
 1, "a",
 2, "b",
 3, "c")
```

Both make this tibble

```
A tibble: 3 x 2
 x y
 <int> <chr>
1 1 a
2 2 b
3 3 c
```

**as\_tibble(x, ...)** Convert a data frame to a tibble.

**enframe(x, name = "name", value = "value")**

Convert a named vector to a tibble. Also **deframe()**.

**is\_tibble(x)** Test whether x is a tibble.



## Reshape Data - Pivot data to reorganize values into a new layout.

table4a

| country | 1999 | 2000 |
|---------|------|------|
| A       | 0.7K | 2K   |
| B       | 37K  | 80K  |
| C       | 212K | 213K |

→

| country | year | cases |
|---------|------|-------|
| A       | 1999 | 0.7K  |
| B       | 1999 | 37K   |
| C       | 1999 | 212K  |
| A       | 2000 | 2K    |
| B       | 2000 | 80K   |
| C       | 2000 | 213K  |

**pivot\_longer**(data, cols, names\_to = "name", values\_to = "value", values\_drop\_na = FALSE)

"Lengthen" data by collapsing several columns into two. Column names move to a new names\_to column and values to a new values\_to column.

```
pivot_longer(table4a, cols = 2:3, names_to = "year",
 values_to = "cases")
```

table2

| country | year | type  | count |
|---------|------|-------|-------|
| A       | 1999 | cases | 0.7K  |
| A       | 1999 | pop   | 19M   |
| A       | 2000 | cases | 2K    |
| A       | 2000 | pop   | 20M   |
| B       | 1999 | cases | 37K   |
| B       | 1999 | pop   | 172M  |
| B       | 2000 | cases | 80K   |
| B       | 2000 | pop   | 174M  |
| C       | 1999 | cases | 212K  |
| C       | 1999 | pop   | 1T    |
| C       | 2000 | cases | 213K  |
| C       | 2000 | pop   | 1T    |

→

| country | year | cases | pop  |
|---------|------|-------|------|
| A       | 1999 | 0.7K  | 19M  |
| A       | 2000 | 2K    | 20M  |
| B       | 1999 | 37K   | 172M |
| B       | 2000 | 80K   | 174M |
| C       | 1999 | 212K  | 1T   |
| C       | 2000 | 213K  | 1T   |

**pivot\_wider**(data, names\_from = "name", values\_from = "value")

The inverse of pivot\_longer(). "Widen" data by expanding two columns into several. One column provides the new column names, the other the values.

```
pivot_wider(table2, names_from = type,
 values_from = count)
```

## Split Cells - Use these functions to split or combine cells into individual, isolated values.

table5

| country | century | year |
|---------|---------|------|
| A       | 19      | 99   |
| A       | 20      | 00   |
| B       | 19      | 99   |
| B       | 20      | 00   |

→

| country | year |
|---------|------|
| A       | 1999 |
| A       | 2000 |
| B       | 1999 |
| B       | 2000 |

**unite**(data, col, ..., sep = "\_", remove = TRUE, na.rm = FALSE) Collapse cells across several columns into a single column.

```
unite(table5, century, year, col = "year", sep = "")
```

table3

| country | year | rate     |
|---------|------|----------|
| A       | 1999 | 0.7K/19M |
| A       | 2000 | 2K/20M   |
| B       | 1999 | 37K/172M |
| B       | 2000 | 80K/174M |

→

| country | year | cases | pop |
|---------|------|-------|-----|
| A       | 1999 | 0.7K  | 19M |
| A       | 2000 | 2K    | 20M |
| B       | 1999 | 37K   | 172 |
| B       | 2000 | 80K   | 174 |

**separate\_wider\_delim**(data, cols, delim, ..., names = NULL, names\_sep = NULL, names\_repair = "check\_unique", too\_few, too\_many, cols\_remove = TRUE) Separate each cell in a column into several columns. Also **separate\_wider\_regex()** and **separate\_wider\_position()**.

```
separate(table3, rate, sep = "/",
 into = c("cases", "pop"))
```

table3

| country | year | rate     |
|---------|------|----------|
| A       | 1999 | 0.7K/19M |
| A       | 2000 | 2K/20M   |
| B       | 1999 | 37K/172M |
| B       | 2000 | 80K/174M |

→

| country | year | cases | pop  |
|---------|------|-------|------|
| A       | 1999 | 0.7K  | 19M  |
| A       | 2000 | 2K    | 20M  |
| B       | 1999 | 37K   | 172M |
| B       | 2000 | 80K   | 174M |

**separate\_longer\_delim**(data, cols, delim, ..., width, keep\_empty) Separate each cell in a column into several rows.

```
separate_longer_delim(table3, rate, sep = "/")
```

## Expand Tables

Create new combinations of variables or identify implicit missing values (combinations of variables not present in the data).

x

| x1 | x2 | x3 |
|----|----|----|
| A  | 1  | 3  |
| B  | 1  | 4  |
| B  | 2  | 3  |

→

| x1 | x2 |
|----|----|
| A  | 1  |
| A  | 2  |
| B  | 1  |
| B  | 2  |

**expand**(data, ...) Create a new tibble with all possible combinations of the values of the variables listed in ... Drop other variables.

```
expand(mtcars, cyl, gear,
 carb)
```

x

| x1 | x2 | x3 |
|----|----|----|
| A  | 1  | 3  |
| B  | 1  | 4  |
| B  | 2  | 3  |

→

| x1 | x2 | x3 |
|----|----|----|
| A  | 1  | 3  |
| A  | 2  | NA |
| B  | 1  | 4  |
| B  | 2  | 3  |

**complete**(data, ..., fill = list()) Add missing possible combinations of values of variables listed in ... Fill remaining variables with NA.

```
complete(mtcars, cyl, gear,
 carb)
```

## Handle Missing Values

Drop or replace explicit missing values (NA).

x

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | NA |
| C  | NA |
| D  | 3  |
| E  | NA |

→

| x1 | x2 |
|----|----|
| A  | 1  |
| D  | 3  |

**drop\_na**(data, ...) Drop rows containing NA's in ... columns.

```
drop_na(x, x2)
```

x

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | NA |
| C  | NA |
| D  | 3  |
| E  | NA |

→

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | 1  |
| C  | 1  |
| D  | 3  |
| E  | 3  |

**fill**(data, ..., .direction = "down") Fill in NA's in ... columns using the next or previous value.

```
fill(x, x2)
```

x

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | NA |
| C  | NA |
| D  | 3  |
| E  | NA |

→

| x1 | x2 |
|----|----|
| A  | 1  |
| B  | 2  |
| C  | 2  |
| D  | 3  |
| E  | 2  |

**replace\_na**(data, replace) Specify a value to replace NA in selected columns.

```
replace_na(x, list(x2 = 2))
```

# Nested Data

A **nested data frame** stores individual tables as a list-column of data frames within a larger organizing data frame. List-columns can also be lists of vectors or lists of varying data types.

Use a nested data frame to:

- Preserve relationships between observations and subsets of data. Preserve the type of the variables being nested (factors and datetimes aren't coerced to character).
- Manipulate many sub-tables at once with **purrr** functions like `map()`, `map2()`, or `pmap()` or with **dplyr** `rowwise()` grouping.

## CREATE NESTED DATA

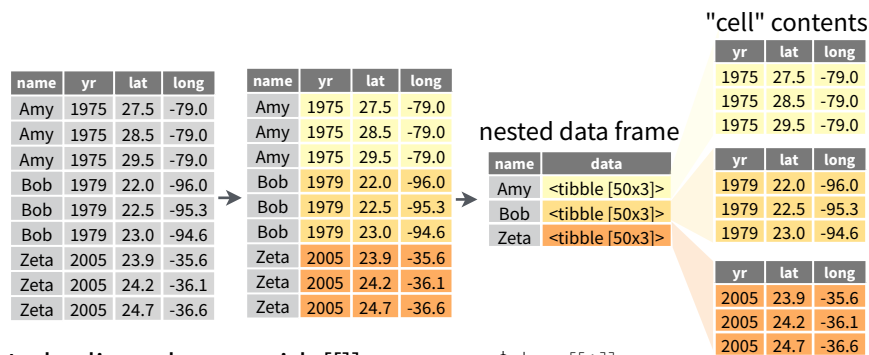
**nest(data, ...)** Moves groups of cells into a list-column of a data frame. Use alone or with **dplyr::group\_by()**:

1. Group the data frame with **group\_by()** and use **nest()** to move the groups into a list-column.

```
n_storms <- storms |>
 group_by(name) |>
 nest()
```

2. Use **nest(new\_col = c(x, y))** to specify the columns to group using **dplyr::select()** syntax.

```
n_storms <- storms |>
 nest(data = c(year:long))
```



Index list-columns with `[[ ]]`. `n_storms$data[[1]]`

## CREATE TIBBLES WITH LIST-COLUMNS

**tibble::tribble(...)** Makes list-columns when needed.

```
tribble(~max, ~seq,
 3, 1:3,
 4, 1:4,
 5, 1:5)
```

| max | seq       |
|-----|-----------|
| 3   | <int [3]> |
| 4   | <int [4]> |
| 5   | <int [5]> |

**tibble::tibble(...)** Saves list input as list-columns.

```
tibble(max = c(3, 4, 5), seq = list(1:3, 1:4, 1:5))
```

**tibble::enframe(x, name="name", value="value")**

Converts multi-level list to a tibble with list-cols.

```
enframe(list('3'=1:3, '4'=1:4, '5'=1:5), 'max', 'seq')
```

## OUTPUT LIST-COLUMNS FROM OTHER FUNCTIONS

**dplyr::mutate()**, **transmute()**, and **summarise()** will output list-columns if they return a list.

```
mtcars |>
 group_by(cyl) |>
 summarise(q = list(quantile(mpg)))
```

## RESHAPE NESTED DATA

**unnest(data, cols, ..., keep\_empty = FALSE)** Flatten nested columns back to regular columns. The inverse of `nest()`.

```
n_storms |> unnest(data)
```

**unnest\_longer(data, col, values\_to = NULL, indices\_to = NULL)**

Turn each element of a list-column into a row.

```
starwars |>
 select(name, films) |>
 unnest_longer(films)
```

| name  | films                |
|-------|----------------------|
| Luke  | The Empire Strik...  |
| Luke  | Revenge of the S...  |
| Luke  | Return of the Jed... |
| C-3PO | The Empire Strik...  |
| C-3PO | Attack of the Cl...  |
| C-3PO | The Phantom M...     |
| R2-D2 | The Empire Strik...  |
| R2-D2 | Attack of the Cl...  |
| R2-D2 | The Phantom M...     |

**unnest\_wider(data, col)** Turn each element of a list-column into a regular column.

```
starwars |>
 select(name, films) |>
 unnest_wider(films, names_sep = "_")
```

| name  | films     |
|-------|-----------|
| Luke  | <chr [5]> |
| C-3PO | <chr [6]> |
| R2-D2 | <chr [7]> |

| name  | films_1       | films_2       | films_3        |
|-------|---------------|---------------|----------------|
| Luke  | The Empire... | Revenge of... | Return of...   |
| C-3PO | The Empire... | Attack of...  | The Phantom... |
| R2-D2 | The Empire... | Attack of...  | The Phantom... |

**hoist(.data, .col, ..., .remove = TRUE)** Selectively pull list components out into their own top-level columns. Uses **purrr::pluck()** syntax for selecting from lists.

```
starwars |>
 select(name, films) |>
 hoist(films, first_film = 1, second_film = 2)
```

| name  | films     |
|-------|-----------|
| Luke  | <chr [5]> |
| C-3PO | <chr [6]> |
| R2-D2 | <chr [7]> |

| name  | first_film    | second_film   | films     |
|-------|---------------|---------------|-----------|
| Luke  | The Empire... | Revenge of... | <chr [3]> |
| C-3PO | The Empire... | Attack of...  | <chr [4]> |
| R2-D2 | The Empire... | Attack of...  | <chr [5]> |

## TRANSFORM NESTED DATA

A vectorized function takes a vector, transforms each element in parallel, and returns a vector of the same length. By themselves vectorized functions cannot work with lists, such as list-columns.

**dplyr::rowwise(.data, ...)** Group data so that each row is one group, and within the groups, elements of list-columns appear directly (accessed with `[[ ]]`, not as lists of length one). **When you use `rowwise()`, dplyr functions will seem to apply functions to list-columns in a vectorized fashion.**



Apply a function to a list-column and **create a new list-column.**

```
n_storms |>
 rowwise() |>
 mutate(n = list(dim(data)))
```

**dim() returns two values per row**

**wrap with list to tell mutate to create a list-column**

Apply a function to a list-column and **create a regular column.**

```
n_storms |>
 rowwise() |>
 mutate(n = nrow(data))
```

**nrow() returns one integer per row**

Collapse **multiple list-columns** into a single list-column.

```
starwars |>
 rowwise() |>
 mutate(transport = list(append(vehicles, starships)))
```

**append() returns a list for each row, so col type must be list**

Apply a function to **multiple list-columns.**

```
starwars |>
 rowwise() |>
 mutate(n_transports = length(c(vehicles, starships)))
```

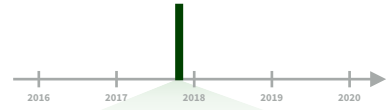
**length() returns one integer per row**

See **purrr** package for more list functions.

# Dates and times with lubridate : : CHEATSHEET



## Date-times



2017-11-28 12:00:00

2017-11-28 12:00:00

A **date-time** is a point on the timeline, stored as the number of seconds since 1970-01-01 00:00:00 UTC

```
dt <- as_datetime(1511870400)
"2017-11-28 12:00:00 UTC"
```

### PARSE DATE-TIMES (Convert strings or numbers to date-times)

1. Identify the order of the year (**y**), month (**m**), day (**d**), hour (**h**), minute (**m**) and second (**s**) elements in your data.
2. Use the function below whose name replicates the order. Each accepts a tz argument to set the time zone, e.g. ymd(x, tz = "UTC").

2017-11-28T14:02:00 ymd\_hms(), ymd\_hm(), ymd\_h().  
ymd\_hms("2017-11-28T14:02:00")

2017-22-12 10:00:00 ydm\_hms(), ydm\_hm(), ydm\_h().  
ydm\_hms("2017-22-12 10:00:00")

11/28/2017 1:02:03 mdy\_hms(), mdy\_hm(), mdy\_h().  
mdy\_hms("11/28/2017 1:02:03")

1 Jan 2017 23:59:59 dmy\_hms(), dmy\_hm(), dmy\_h().  
dmy\_hms("1 Jan 2017 23:59:59")

20170131 ymd(), ydm(). ymd(20170131)

July 4th, 2000 mdy(), myd(). mdy("July 4th, 2000")

4th of July '99 dmy(), dym(). dmy("4th of July '99")

2001: Q3 yq() Q for quarter. yq("2001: Q3")

07-2020 my(), ym(). my("07-2020")

2:01 hms::hms() Also lubridate::hms(),  
hm() and ms(), which return  
periods.\* hms::hms(seconds = 0,  
minutes = 1, hours = 2)

2017.5 date\_decimal(decimal, tz = "UTC")  
date\_decimal(2017.5)

now(tzone = "") Current time in tz  
(defaults to system tz). now()

today(tzone = "") Current date in a  
tz (defaults to system tz). today()

fast\_strptime() Faster strptime.  
fast\_strptime("9/1/01", "%y/%m/%d")

parse\_date\_time() Easier strptime.  
parse\_date\_time("09-01-01", "ymd")

2017-11-28

A **date** is a day stored as the number of days since 1970-01-01

```
d <- as_date(17498)
"2017-11-28"
```

### GET AND SET COMPONENTS

Use an accessor function to get a component.  
Assign into an accessor function to change a component in place.

```
d ## "2017-11-28"
day(d) ## 28
day(d) <- 1
d ## "2017-11-01"
```

12:00:00

An **hms** is a **time** stored as the number of seconds since 00:00:00

```
t <- hms::as_hms(85)
00:01:25
```

2018-01-31 11:59:59

date(x) Date component. date(dt)

2018-01-31 11:59:59

year(x) Year. year(dt)  
isoyear(x) The ISO 8601 year.  
epiyear(x) Epidemiological year.

2018-01-31 11:59:59

month(x, label, abbr) Month.  
month(dt)

2018-01-31 11:59:59

day(x) Day of month. day(dt)  
wday(x, label, abbr) Day of week.  
qday(x) Day of quarter.

2018-01-31 11:59:59

hour(x) Hour. hour(dt)

2018-01-31 11:59:59

minute(x) Minutes. minute(dt)

2018-01-31 11:59:59

second(x) Seconds. second(dt)

2018-01-31 11:59:59 UTC

tz(x) Time zone. tz(dt)

week(x) Week of the year. week(dt)  
isoweek() ISO 8601 week.  
epiweek() Epidemiological week.

quarter(x) Quarter. quarter(dt)

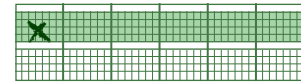
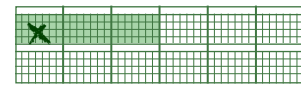
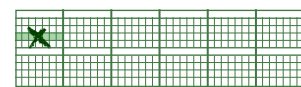
semester(x, with\_year = FALSE)  
Semester. semester(dt)

am(x) Is it in the am? am(dt)  
pm(x) Is it in the pm? pm(dt)

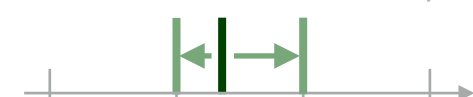
dst(x) Is it daylight savings? dst(d)

leap\_year(x) Is it a leap year?  
leap\_year(d)

update(object, ..., simple = FALSE)  
update(dt, mday = 2, hour = 1)



## Round Date-times



floor\_date(x, unit = "second")  
Round down to nearest unit.  
floor\_date(dt, unit = "month")

round\_date(x, unit = "second")  
Round to nearest unit.  
round\_date(dt, unit = "month")

ceiling\_date(x, unit = "second",  
change\_on\_boundary = NULL)  
Round up to nearest unit.  
ceiling\_date(dt, unit = "month")

Valid units are second, minute, hour, day, week, month, bimonth, quarter, season, halfyear and year.

rollback(dates, roll\_to\_first = FALSE, preserve\_hms = TRUE) Roll back to last day of previous month. Also **rollforward()**. rollback(dt)

## Stamp Date-times

stamp() Derive a template from an example string and return a new function that will apply the template to date-times. Also **stamp\_date()** and **stamp\_time()**.

1. Derive a template, create a function  
sf <- stamp("Created Sunday, Jan 17, 1999 3:34")
2. Apply the template to dates  
sf(ymd("2010-04-05"))  
## [1] "Created Monday, Apr 05, 2010 00:00"

Tip: use a  
date with  
day > 12

## Time Zones

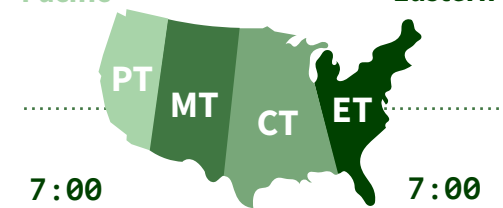
R recognizes ~600 time zones. Each encodes the time zone, Daylight Savings Time, and historical calendar variations for an area. R assigns one time zone per vector.

Use the **UTC** time zone to avoid Daylight Savings.

OlsonNames() Returns a list of valid time zone names. OlsonNames()

Sys.timezone() Gets current time zone.

4:00 Pacific 5:00 Mountain 6:00 Central 7:00 Eastern



7:00 Pacific 7:00 Mountain 7:00 Central 7:00 Eastern

with\_tz(time, tzone = "") Get the **same date-time** in a new time zone (a new clock time). Also **local\_time(dt, tz, units)**. with\_tz(dt, "US/Pacific")

force\_tz(time, tzone = "") Get the **same clock time** in a new time zone (a new date-time). Also **force\_tzs()**. force\_tz(dt, "US/Pacific")

# Math with Date-times — Lubridate provides three classes of timespans to facilitate math with dates and date-times.



Math with date-times relies on the **timeline**, which behaves inconsistently. Consider how the timeline behaves during:

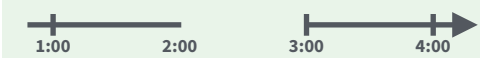
## A normal day

```
nor <- ymd_hms("2018-01-01 01:30:00", tz="US/Eastern")
```



## The start of daylight savings (spring forward)

```
gap <- ymd_hms("2018-03-11 01:30:00", tz="US/Eastern")
```



## The end of daylight savings (fall back)

```
lap <- ymd_hms("2018-11-04 00:30:00", tz="US/Eastern")
```



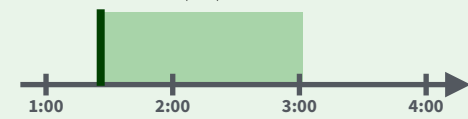
## Leap years and leap seconds

```
leap <- ymd("2019-03-01")
```

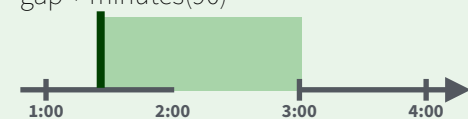


**Periods** track changes in clock times, which ignore time line irregularities.

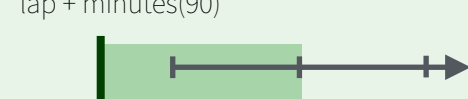
nor + minutes(90)



gap + minutes(90)



lap + minutes(90)

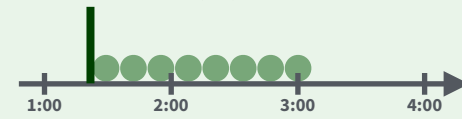


leap + years(1)



**Durations** track the passage of physical time, which deviates from clock time when irregularities occur.

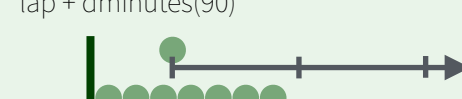
nor + dminutes(90)



gap + dminutes(90)



lap + dminutes(90)

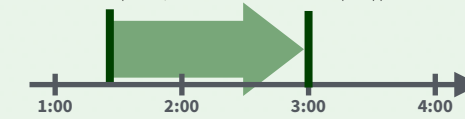


leap + dyears(1)

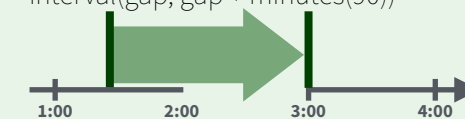


**Intervals** represent specific intervals of the timeline, bounded by start and end date-times.

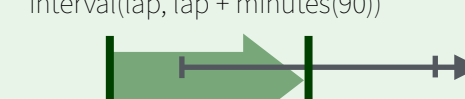
interval(nor, nor + minutes(90))



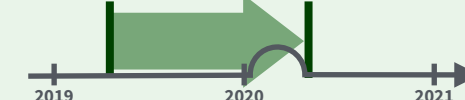
interval(gap, gap + minutes(90))



interval(lap, lap + minutes(90))



interval(leap, leap + years(1))



Not all years are 365 days due to **leap days**.

Not all minutes are 60 seconds due to **leap seconds**.

It is possible to create an imaginary date by adding **months**, e.g. February 31st

```
jan31 <- ymd(20180131)
jan31 + months(1)
NA
```

**%m+%** and **%m-%** will roll imaginary dates to the last day of the previous month.

```
jan31 %m+% months(1)
"2018-02-28"
```

**add\_with\_rollback(e1, e2, roll\_to\_first = TRUE)** will roll imaginary dates to the first day of the new month.

```
add_with_rollback(jan31, months(1),
 roll_to_first = TRUE)
"2018-03-01"
```

## PERIODS

Add or subtract periods to model events that happen at specific clock times, like the NYSE opening bell.

Make a period with the name of a time unit **pluralized**, e.g.

```
p <- months(3) + days(12)
```

```
p
"3m 12d 0H 0M 0S"
```

Number of months Number of days etc.

**years(x = 1)** x years.

**months(x = 1)** x months.

**weeks(x = 1)** x weeks.

**days(x = 1)** x days.

**hours(x = 1)** x hours.

**minutes(x = 1)** x minutes.

**seconds(x = 1)** x seconds.

**milliseconds(x = 1)** x milliseconds.

**microseconds(x = 1)** x microseconds.

**nanoseconds(x = 1)** x nanoseconds.

**picoseconds(x = 1)** x picoseconds.

**period(num = NULL, units = "second", ...)**  
An automation friendly period constructor.  
period(5, unit = "years")

**as.period(x, unit)** Coerce a timespan to a period, optionally in the specified units.  
Also **is.period()**. as.period(p)

**period\_to\_seconds(x)** Convert a period to the "standard" number of seconds implied by the period. Also **seconds\_to\_period()**.  
period\_to\_seconds(p)

## DURATIONS

Add or subtract durations to model physical processes, like battery life. Durations are stored as seconds, the only time unit with a consistent length. **Diffimes** are a class of durations found in base R.

Make a duration with the name of a period prefixed with a **d**, e.g.

```
dd <- ddays(14)
```

```
dd
```

```
"1209600s (~2 weeks)"
```

Exact length in seconds Equivalent in common units

**dyears(x = 1)** 31536000x seconds.

**dmonths(x = 1)** 2629800x seconds.

**dweeks(x = 1)** 604800x seconds.

**ddays(x = 1)** 86400x seconds.

**dhours(x = 1)** 3600x seconds.

**dminutes(x = 1)** 60x seconds.

**dseconds(x = 1)** x seconds.

**dmilliseconds(x = 1)**  $x \times 10^{-3}$  seconds.

**dmicroseconds(x = 1)**  $x \times 10^{-6}$  seconds.

**dnanoseconds(x = 1)**  $x \times 10^{-9}$  seconds.

**dpicoseconds(x = 1)**  $x \times 10^{-12}$  seconds.

**duration(num = NULL, units = "second", ...)**  
An automation friendly duration constructor. duration(5, unit = "years")

**as.duration(x, ...)** Coerce a timespan to a duration. Also **is.duration()**, **is.diffime()**.  
as.duration(i)

**make\_diffime(x)** Make diffime with the specified number of units.  
make\_diffime(99999)

## INTERVALS

Divide an interval by a duration to determine its physical length, divide an interval by a period to determine its implied length in clock time.

Make an interval with **interval()** or **%--%**, e.g.

```
i <- interval(ymd("2017-01-01"), d)
```

```
j <- d %--% ymd("2017-12-31")
```

```
2017-01-01 UTC--2017-11-28 UTC
```

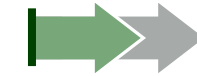
```
2017-11-28 UTC--2017-12-31 UTC
```



**a %within% b** Does interval or date-time *a* fall within interval *b*? now() %within% i



**int\_start(int)** Access/set the start date-time of an interval. Also **int\_end()**. int\_start(i) <- now(); int\_start(i)



**int\_aligns(int1, int2)** Do two intervals share a boundary? Also **int\_overlaps()**. int\_aligns(i, j)



**int\_diff(times)** Make the intervals that occur between the date-times in a vector.  
v <- c(dt, dt + 100, dt + 1000); int\_diff(v)



**int\_flip(int)** Reverse the direction of an interval. Also **int\_standardize()**. int\_flip(i)



**int\_length(int)** Length in seconds. int\_length(i)



**int\_shift(int, by)** Shifts an interval up or down the timeline by a timespan. int\_shift(i, days(-1))

**as.interval(x, start, ...)** Coerce a timespan to an interval with the start date-time. Also **is.interval()**. as.interval(days(1), start = now())

# Apply functions with purrr : : CHEATSHEET



## Map Functions

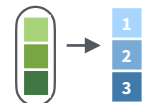
### ONE LIST

**map(.x, .f, ...)** Apply a function to each element of a list or vector, and return a list.

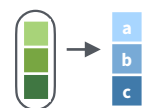
```
x <- list(a = 1:10, b = 11:20, c = 21:30)
l1 <- list(x = c("a", "b"), y = c("c", "d"))
map(l1, sort, decreasing = TRUE)
```



**map\_dbl(.x, .f, ...)**  
Return a double vector.  
`map_dbl(x, mean)`



**map\_int(.x, .f, ...)**  
Return an integer vector.  
`map_int(x, length)`



**map\_chr(.x, .f, ...)**  
Return a character vector.  
`map_chr(l1, paste, collapse = "")`



**map\_lgl(.x, .f, ...)**  
Return a logical vector.  
`map_lgl(x, is.integer)`



**map\_vec(.x, .f, ...)**  
Return a vector that is of the simplest common type.  
`map_vec(l1, paste, collapse = "")`

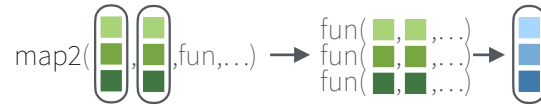


**walk(.x, .f, ...)** Trigger side effects, return invisibly.  
`walk(x, print)`

### TWO LISTS

**map2(.x, .y, .f, ...)** Apply a function to pairs of elements from two lists or vectors, return a list.

```
y <- list(1, 2, 3); z <- list(4, 5, 6); l2 <- list(x = "a", y = "z")
map2(x, y, \(x, y) x*y)
```



**map2\_dbl(.x, .y, .f, ...)** Return a double vector.  
`map2_dbl(y, z, ~.x / .y)`



**map2\_int(.x, .y, .f, ...)** Return an integer vector.  
`map2_int(y, z, `+`)`



**map2\_chr(.x, .y, .f, ...)** Return a character vector.  
`map2_chr(l1, l2, paste, collapse = ",", sep = ":")`



**map2\_lgl(.x, .y, .f, ...)** Return a logical vector.  
`map2_lgl(l2, l1, `%%in%%`)`



**map2\_vec(.x, .f, ...)**  
Return a vector that is of the simplest common type.  
`map2_vec(l1, l2, paste, collapse = ",", sep = ":")`



**walk2(.x, .y, .f, ...)** Trigger side effects, return invisibly.  
`walk2(objs, paths, save)`

### MANY LISTS

**pmap(.l, .f, ...)** Apply a function to groups of elements from a list of lists or vectors, return a list.

```
pmap(
 list(x, y, z),
 function(first, second, third) first * (second + third)
)
```



**pmap\_dbl(.l, .f, ...)**  
Return a double vector.  
`pmap_dbl(list(y, z), ~.x / .y)`



**pmap\_int(.l, .f, ...)**  
Return an integer vector.  
`pmap_int(list(y, z), `+`)`



**pmap\_chr(.l, .f, ...)**  
Return a character vector.  
`pmap_chr(list(l1, l2), paste, collapse = ",", sep = ":")`



**pmap\_lgl(.l, .f, ...)**  
Return a logical vector.  
`pmap_lgl(list(l2, l1), `%%in%%`)`



**pmap\_vec(.l, .f, ...)**  
Return a vector that is of the simplest common type.  
`pmap_vec(list(l1, l2), paste, collapse = ",", sep = ":")`



**pwalk(.l, .f, ...)** Trigger side effects, return invisibly.  
`pwalk(list(objs, paths), save)`

**imap(.x, .f, ...)** is shorthand for **map2(.x, names(.x), .f)** or **map2(.x, seq\_along(.x), .f)** depending on whether **.x** is named or not.

## Function Shortcuts

Use **\(x)** with functions like **map()** that have single arguments.

**map(l, \(x) x + 2)**  
becomes  
**map(l, function(x) x + 2)**

Use **\(x, y)** with functions like **map2()** that have two arguments.

**map2(l, p, \(x, y) x + y)**  
becomes  
**map2(l, p, function(l, p) l + p)**

Use **\(x, y, z)** etc with functions like **pmap()** that have many arguments.

**pmap(list(x, y, z), \(x, y, z) x + y / z)**  
becomes  
**pmap(list(x, y, z), function(x, y, z) x \* (y + z))**

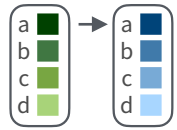
Use **\(x, y)** with functions like **imap()**. **.x** will get the list value and **.y** will get the index, or name if available.

**imap(list("a", "b", "c"), \(x, y) paste0(y, ": ", x))**  
outputs **"index: value"** for each item

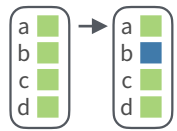
Use a **string** or an **integer** with any map function to index list elements by name or position. **map(l, "name")** becomes **map(l, function(x) x[["name"]])**



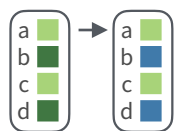
## Modify



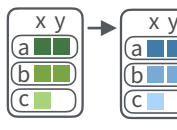
**modify(.x, .f, ...)** Apply a function to each element. Also **modify2()**, and **imodify()**.  
`modify(x, ~.+ 2)`



**modify\_at(.x, .at, .f, ...)** Apply a function to selected elements. Also **map\_at()**.  
`modify_at(x, "b", ~.+ 2)`



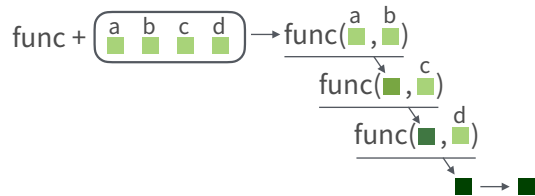
**modify\_if(.x, .p, .f, ...)** Apply a function to elements that pass a test. Also **map\_if()**.  
`modify_if(x, is.numeric, ~.+2)`



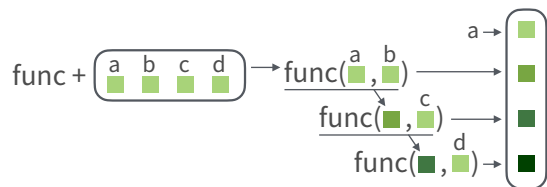
**modify\_depth(.x, .depth, .f, ...)** Apply function to each element at a given level of a list. Also **map\_depth()**.  
`modify_depth(x, 1, ~.+ 2)`

## Reduce

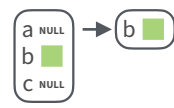
**reduce(.x, .f, ..., .init, .dir = c("forward", "backward"))** Apply function recursively to each element of a list or vector. Also **reduce2()**.  
`reduce(x, sum)`



**accumulate(.x, .f, ..., .init)** Reduce a list, but also return intermediate results. Also **accumulate2()**.  
`accumulate(x, sum)`



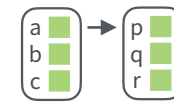
## Vectors



**compact(.x, .p = identity)** Discard empty elements.  
`compact(x)`

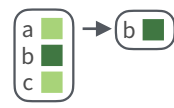


**keep\_at(x, at)** Keep/discard elements based by name or position. Conversely, **discard\_at()**.  
`keep_at(x, "a")`

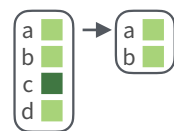


**set\_names(x, nm = x)** Set the names of a vector/list directly or with a function.  
`set_names(x, c("p", "q", "r"))`  
`set_names(x, tolower)`

## Predicate functions



**keep(.x, .p, ...)** Keep elements that pass a logical test. Conversely, **discard()**.  
`keep(x, is.numeric)`



**head\_while(.x, .p, ...)** Return head elements until one does not pass. Also **tail\_while()**.  
`head_while(x, is.character)`



**detect(.x, .f, ..., .dir = c("forward", "backward"), .right = NULL, .default = NULL)** Find first element to pass.  
`detect(x, is.character)`



**detect\_index(.x, .f, ..., .dir = c("forward", "backward"), .right = NULL)** Find index of first element to pass.  
`detect_index(x, is.character)`



**every(.x, .p, ...)** Do all elements pass a test?  
`every(x, is.character)`



**some(.x, .p, ...)** Do some elements pass a test?  
`some(x, is.character)`

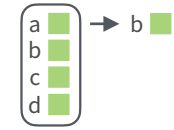


**none(.x, .p, ...)** Do no elements pass a test?  
`none(x, is.character)`

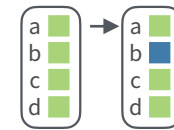


**has\_element(.x, .y)** Does a list contain an element?  
`has_element(x, "foo")`

## Pluck



**pluck(.x, ..., .default=NULL)** Select an element by name or index. Also **attr\_getter()** and **chuck()**.  
`pluck(x, "b")`  
`x |> pluck("b")`

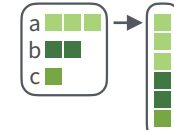


**assign\_in(x, where, value)** Assign a value to a location using pluck selection.  
`assign_in(x, "b", 5)`  
`x |> assign_in("b", 5)`



**modify\_in(.x, .where, .f)** Apply a function to a value at a selected location.  
`modify_in(x, "b", abs)`  
`x |> modify_in("b", abs)`

## Reshape



**list\_flatten(.x)** Remove a level of indexes from a list.  
`list_flatten(x)`

## List-Columns

| max | seq       |
|-----|-----------|
| 3   | <int [3]> |
| 4   | <int [4]> |
| 5   | <int [5]> |

**List-columns** are columns of a data frame where each element is a list or vector instead of an atomic value. Columns can also be lists of data frames. See **tidyr** for more about nested data and list columns.

### WORK WITH LIST-COLUMNS

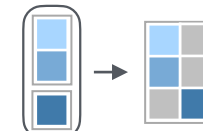
Manipulate list-columns like any other kind of column, using **dplyr** functions like **mutate()**. Because each element is a list, use **map functions** within a column function to manipulate each element.

## Concatenate

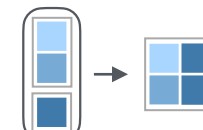
```
x1 <- list(a = 1, b = 2, c = 3)
x2 <- list(
 a = data.frame(x = 1:2),
 b = data.frame(y = "a")
)
```



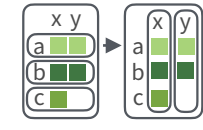
**list\_c(x)** Combines elements into a vector by concatenating them together.  
`list_c(x1)`



**list\_rbind(x)** Combines elements into a data frame by row-binding them together.  
`list_rbind(x2)`



**list\_cbind(x)** Combines elements into a data frame by column-binding them together.  
`list_cbind(x2)`



**list\_transpose(.l, .names = NULL)** Transposes the index order in a multi-level list.  
`list_transpose(x)`

**map(), map2(), or pmap()** return lists and will create new **list-columns**.

```
starwars |>
 transmute(ships = map2(vehicles,
 starships,
 append))
```

list function, return list  
list-columns  
column function

Suffixed map functions like **map\_int()** return an atomic data type and will **simplify list-columns into regular columns**.

```
starwars |>
 mutate(n_films = map_int(films, length))
```

list function, return int  
column function  
list-column

# rmarkdown :: CHEATSHEET

## What is rmarkdown?

- .Rmd files** • Develop your code and ideas side-by-side in a single document. Run code as individual chunks or as an entire document.
- Dynamic Documents** • Knit together plots, tables, and results with narrative text. Render to a variety of formats like HTML, PDF, MS Word, or MS Powerpoint.
- Reproducible Research** • Upload, link to, or attach your report to share. Anyone can read or run your code to reproduce your work.

## Workflow

- 1 Open a **new .Rmd file** in the RStudio IDE by going to **File > New File > R Markdown**.
- 2 **Embed code** in chunks. Run code by line, by chunk, or all at once.
- 3 **Write text** and add tables, figures, images, and citations. Format with Markdown syntax or the RStudio Visual Markdown Editor.
- 4 **Set output format(s) and options** in the YAML header. Customize themes or add parameters to execute or add interactivity with Shiny.
- 5 **Save and render** the whole document. Knit periodically to preview your work as you write.
- 6 **Share your work!**

## Embed Code with knitr

**CODE CHUNKS**  
Surround code chunks with ````{r}` and ````` or use the Insert Code Chunk button. Add a chunk label and/or chunk options inside the curly braces after `r`.

```
```{r chunk-label, include=FALSE}
summary(mtcars)
```
```

**SET GLOBAL OPTIONS**  
Set options for the entire document in the first chunk.

```
```{r include=FALSE}
knitr::opts_chunk$set(message = FALSE)
```
```

**INLINE CODE**  
Insert ``r <code>`` into text sections. Code is evaluated at render and results appear as text.

"Built with ``r getRversion()``" -> "Built with 4.1.0"



**1. New File**

**5. Save and Render**

**6. Share**

**4. Set Output Format(s) and Options**

**3. Write Text**

**2. Embed Code**

Source Editor

set preview location

insert code chunk

go to code chunk

run code chunk(s)

show outline

run all previous chunks

run current chunk

modify chunk options

Output created: report.html

**VISUAL EDITOR**

add/edit attributes

style options

insert citations

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents.

| OPTION                 | DEFAULT   | EFFECTS                                                                                                             |
|------------------------|-----------|---------------------------------------------------------------------------------------------------------------------|
| echo                   | TRUE      | display code in output document                                                                                     |
| error                  | FALSE     | TRUE (display error messages in doc)<br>FALSE (stop render when error occurs)                                       |
| eval                   | TRUE      | run code in chunk                                                                                                   |
| include                | TRUE      | include chunk in doc after running                                                                                  |
| message                | TRUE      | display code messages in document                                                                                   |
| warning                | TRUE      | display code warnings in document                                                                                   |
| results                | "markup"  | "asis" (passthrough results)<br>"hide" (don't display results)<br>"hold" (put all results below all code)           |
| fig.align              | "default" | "left", "right", or "center"                                                                                        |
| fig.alt                | NULL      | alt text for a figure                                                                                               |
| fig.cap                | NULL      | figure caption as a character string                                                                                |
| fig.path               | "figure/" | prefix for generating figure file paths                                                                             |
| fig.width & fig.height | 7         | plot dimensions in inches                                                                                           |
| collapse               | FALSE     | rescales output width, e.g. "75%", "300px"                                                                          |
| comment                | "###"     | collapse all sources & output into a single block                                                                   |
| child                  | NULL      | prefix for each line of results                                                                                     |
| purl                   | TRUE      | files(s) to knit and then include<br>include or exclude a code chunk when extracting source code with knitr::purl() |

See more options and defaults by running `str(knitr::opts_chunk$get())`

**RENDERED OUTPUT**

file path to output document

find in document

publish to [rpubs.com](#), [shinyapps.io](#), Posit Connect

reload document

Document Title

Author Name

- R Markdown
- Including Plots

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

```
summary(cars)
```

| ##         | speed  | dist           |
|------------|--------|----------------|
| ## Min.    | : 4.0  | Min. : 2.00    |
| ## 1st Qu. | : 12.0 | 1st Qu.: 26.00 |
| ## Median  | : 15.0 | Median : 36.00 |
| ## Mean    | : 15.4 | Mean : 42.98   |
| ## 3rd Qu. | : 19.0 | 3rd Qu.: 56.00 |
| ## Max.    | : 25.0 | Max. : 120.00  |

## Insert Citations

Create citations from a bibliography file, a Zotero library, or from DOI references.

### BUILD YOUR BIBLIOGRAPHY

- Add BibTeX or CSL bibliographies to the YAML header.
- If Zotero is installed locally, your main library will automatically be available.
- Add citations by DOI by searching "from DOI" in the **Insert Citation** dialog.

### INSERT CITATIONS

- Access the **Insert Citations** dialog in the Visual Editor by clicking the **@** symbol in the toolbar or by clicking **Insert > Citation**.
- Add citations with markdown syntax by typing `[@cite]` or `@cite`.

## Insert Tables

Output data frames as tables using `kable(data, caption)`.

```
```{r}
data <- faithful[1:4, ]
knitr::kable(data,
  caption = "Table with kable")
```
```

Table with kable

| eruptions | waiting |
|-----------|---------|
| 3.600     | 79      |
| 1.800     | 54      |
| 3.333     | 74      |
| 2.283     | 62      |

Other table packages include **flextable**, **gt**, and **kableExtra**.

## Write with Markdown

The syntax on the left renders as the output on the right.

- Plain text.
- End a line with two spaces to start a new paragraph.
- Also end with a backslash to make a new line.
- `*italics*` and `**bold**`
- `superscript^2`/`subscript~2`
- `~~strikethrough~~`
- escaped: `\* \_ \\`
- endash: `--`, emdash: `---`
- Plain text.
- End a line with two spaces to start a new paragraph.
- Also end with a backslash to make a new line.
- italics* and **bold**
- `superscript^2`/`subscript~2`
- ~~strikethrough~~
- escaped: `* _ \`
- endash: `-`, emdash: `—`

- # Header 1
- ## Header 2
- ...
- ##### Header 6
- Header 1
- Header 2
- Header 6
- unordered list
  - item 2
    - item 2a (indent 1 tab)
    - item 2b
1. ordered list
  2. item 2
    - item 2a (indent 1 tab)
    - item 2b

<link url>

[This is a link.](link url)

[This is another link][id].

At the end of the document:

[id]: link url

![Caption](image.png)

or ![Caption][id2]

At the end of the document:

[id2]: image.png

http://www.posit.co/

This is a link.

This is another link.

Caption.

verbatim code

multiple lines of verbatim code

> block quotes

equation:  $e^{i\pi} + 1 = 0$

equation block:

$$E = mc^2$$

horizontal rule:

horizontal rule:

| Right | Left | Default | Center |
|-------|------|---------|--------|
| 12    | 12   | 12      | 12     |
| 123   | 123  | 123     | 123    |
| 1     | 1    | 1       | 1      |

**HTML Tabsets**

```
Results {tabset}
Plots
text

Tables
more text
```

Results

Plots

Tables

text





# Set Output Formats and their Options in YAML

Use the document's YAML header to set an **output format** and customize it with **output options**.

```

title: "My Document"
author: "Author Name"
output:
 html_document:
 toc: TRUE

```

Indent format 2 characters,  
indent options 4 characters

| OUTPUT FORMAT                                                                                                    | CREATES                      |
|------------------------------------------------------------------------------------------------------------------|------------------------------|
| html_document                                                                                                    | .html                        |
| pdf_document*                                                                                                    | .pdf                         |
| word_document                                                                                                    | Microsoft Word (.docx)       |
| powerpoint_presentation                                                                                          | Microsoft Powerpoint (.pptx) |
| odt_document                                                                                                     | OpenDocument Text            |
| rtf_document                                                                                                     | Rich Text Format             |
| md_document                                                                                                      | Markdown                     |
| github_document                                                                                                  | Markdown for Github          |
| ioslides_presentation                                                                                            | ioslides HTML slides         |
| slidy_presentation                                                                                               | Slidy HTML slides            |
| beamer_presentation*                                                                                             | Beamer slides                |
| * Requires LaTeX, use <code>tinytex::install_tinytex()</code>                                                    |                              |
| Also see <code>flexdashboard</code> , <code>bookdown</code> , <code>distill</code> , and <code>blogdown</code> . |                              |

| IMPORTANT OPTIONS   | DESCRIPTION                                                                            | HTML | PDF | MS Word | MS PPT |
|---------------------|----------------------------------------------------------------------------------------|------|-----|---------|--------|
| anchor_sections     | Show section anchors on mouse hover (TRUE or FALSE)                                    | X    |     |         |        |
| citation_package    | The LaTeX package to process citations ("default", "natbib", "biblatex")               |      | X   |         |        |
| code_download       | Give readers an option to download the .Rmd source code (TRUE or FALSE)                | X    |     |         |        |
| code_folding        | Let readers to toggle the display of R code ("none", "hide", or "show")                | X    |     |         |        |
| css                 | CSS or SCSS file to use to style document (e.g. "style.css")                           | X    |     |         |        |
| dev                 | Graphics device to use for figure output (e.g. "png", "pdf")                           | X    | X   |         |        |
| df_print            | Method for printing data frames ("default", "kable", "tibble", "paged")                | X    | X   | X       | X      |
| fig_caption         | Should figures be rendered with captions (TRUE or FALSE)                               | X    | X   | X       | X      |
| highlight           | Syntax highlighting ("tango", "pygments", "kate", "zenburn", "textmate")               | X    | X   | X       |        |
| includes            | File of content to place in doc ("in_header", "before_body", "after_body")             | X    | X   |         |        |
| keep_md             | Keep the Markdown .md file generated by knitting (TRUE or FALSE)                       | X    | X   | X       | X      |
| keep_tex            | Keep the intermediate TEX file used to convert to PDF (TRUE or FALSE)                  | X    |     |         |        |
| latex_engine        | LaTeX engine for producing PDF output ("pdflatex", "xelatex", or "lualatex")           | X    |     |         |        |
| reference_docx/_doc | docx/pptx file containing styles to copy in the output (e.g. "file.docx", "file.pptx") |      | X   | X       |        |
| theme               | Theme options (see Bootswatch and Custom Themes below)                                 | X    |     |         |        |
| toc                 | Add a table of contents at start of document (TRUE or FALSE)                           | X    | X   | X       | X      |
| toc_depth           | The lowest level of headings to add to table of contents (e.g. 2, 3)                   | X    | X   | X       | X      |
| toc_float           | Float the table of contents to the left of the main document content (TRUE or FALSE)   | X    |     |         |        |

Use `?<output format>` to see all of a format's options, e.g. `?html_document`

## Render

When you render a document, rmarkdown:

1. Runs the code and embeds results and text into an .md file with knitr.
2. Converts the .md file into the output format with Pandoc.



**Save**, then **Knit** to preview the document output. The resulting HTML/PDF/MS Word/etc. document will be created and saved in the same directory as the .Rmd file.

Use `rmarkdown::render()` to render/knit in the R console. See `?render` for available options.

## Share

**Publish on Posit Connect**

to share R Markdown documents securely, schedule automatic updates, and interact with parameters in real-time. [posit.co/products/enterprise/connect](https://posit.co/products/enterprise/connect).



## More Header Options

### PARAMETERS

Parameterize your documents to reuse with new inputs (e.g., data, values, etc.).

1. **Add parameters** in the header as sub-values of params.

```

params:
 state: "hawaii"

```
2. **Call parameters** in code using `params$<name>`.

```
```{r}
data <- df[, params$state]
summary(data)
```
```
3. **Set parameters** with Knit with Parameters or the params argument of `render()`.

### REUSABLE TEMPLATES

1. **Create a new package** with a `inst/rmarkdown/templates` directory.
2. **Add a folder** containing **template.yaml** (below) and **skeleton.Rmd** (template contents).

```

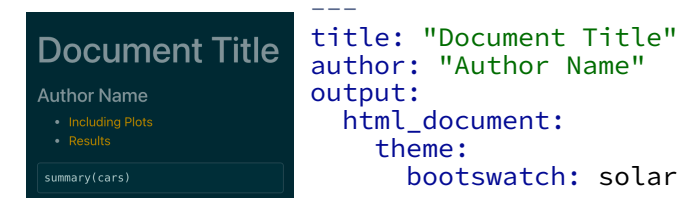
name: "My Template"

```
3. **Install** the package to access template by going to **File > New R Markdown > From Template**.

### BOOTSWATCH THEMES

Customize HTML documents with Bootswatch themes from the **bslib** package using the theme output option.

Use `bslib::bootswatch_themes()` to list available themes.



### CUSTOM THEMES

Customize individual HTML elements using bslib variables. Use `?bs_theme` to see more variables.

```

output:
 html_document:
 theme:
 bg: "#121212"
 fg: "#E4E4E4"
 base_font:
 google: "Prompt"

```

More on **bslib** at [pkgs.rstudio.com/bslib/](https://pkgs.rstudio.com/bslib/).

### STYLING WITH CSS AND SCSS

Add CSS and SCSS to your document by adding a path to a file with the **css** option in the YAML header.

```

title: "My Document"
author: "Author Name"
output:
 html_document:
 css: "style.css"

```

Apply CSS styling by writing HTML tags directly or:

- Use markdown to apply style attributes inline.

**Bracketed Span**  
A `[green]{.my-color}` word.

A **green** word.

**Fenced Div**  
`:: { .my-color }`  
All of these words are green.

All of these words are green.

- Use the Visual Editor. Go to **Format > Div/Span** and add CSS styling directly with Edit Attributes.

`.my-css-tag`

This is a div with some text in it.

### INTERACTIVITY

Turn your report into an interactive Shiny document in 4 steps:

1. Add **runtime: shiny** to the YAML header.
2. Call Shiny input functions to embed input objects.
3. Call Shiny render functions to embed reactive output.
4. Render with `rmarkdown::run()` or click **Run Document** in RStudio IDE.

```

output: html_document
runtime: shiny

```

```
```{r, echo = FALSE}
numericInput("n",
  "How many cars?", 5)

renderTable({
  head(cars, input$n)
})
```

How many cars?		
	5	
	speed	dist
1	4.00	2.00
2	4.00	10.00
3	7.00	4.00
4	7.00	22.00
5	8.00	16.00

Also see Shiny Prerendered for better performance. rmarkdown.rstudio.com/authoring_shiny_prerendered.

Embed a complete app into your document with `shiny::shinyAppDir()`. More at bookdown.org/yihui/rmarkdown/shiny-embedded.html.



Shiny : : CHEATSHEET



Build an app

A **Shiny** app is a web page (**ui**) connected to a computer running a live R session (**server**).



Users can manipulate the UI, which will cause the server to update the UI's displays (by running R code).

Build with AI assistance:
gallery.shinyapps.io/assistant

Get inspiration & examples:

- shiny.posit.co/r/gallery
- shinylive.io/r/examples
- `runExample()` in R console

The UI is a collection of input, output, and layout elements

The server determines how to render outputs given inputs

An **app** is a combination of UI and server logic

```
# app.R
library(shiny)

ui <- bslib::page_fluid(
  sliderInput(
    "n", "Sample size", 0, 100, 25
  ),
  plotOutput("hist")
)

server <- function(input, output) {
  output$hist <- renderPlot({
    hist(rnorm(input$n))
  })
}

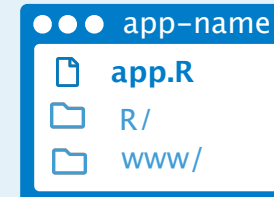
shinyApp(ui, server)
```

Create `*Input()` UI and read with `input$<id>`

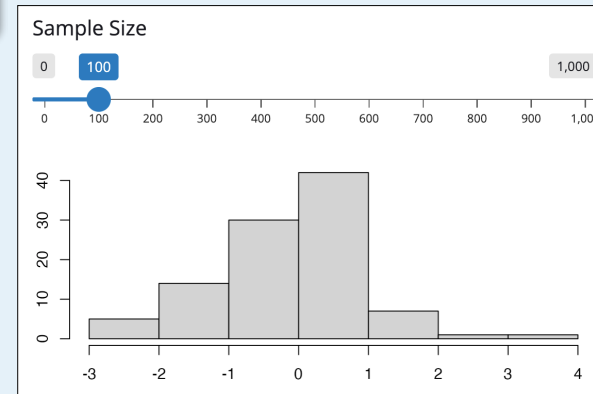
Match `*Output()` with `render*()` to add R output

Save `shinyApp()` to **app.R**

Optionally include supporting code, images, etc. in `R/` and `www/` folders



Launch an **app.R** with `runApp("path/to/app-name")`.



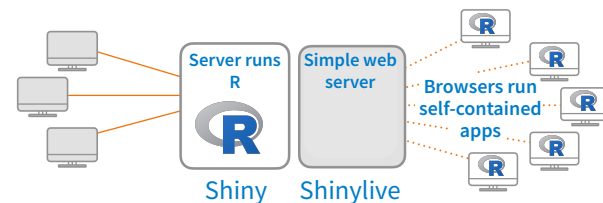
Share

Share your app in four ways:

1. **Host it on shinyapps.io**, a cloud based service from Posit. To deploy Shiny apps:
 - Create a free or professional account at shinyapps.io
 - Click the Publish icon in RStudio IDE, or run: `rsconnect::deployApp("path/to/app-name")`
2. **Purchase Posit Connect**, a publishing platform for R and Python.
posit.co/connect
3. **Host your own Shiny Server**
posit.co/products/open-source/shinyserver
4. **Export to shinylive**, a technology for running apps entirely in the browser.
posit-dev.github.io/r-shinylive

Shinylive

Shinylive apps use WebAssembly to run entirely in a browser—no need for a server to run R.



- Edit and/or host apps at shinylive.io/r
- Export an app to Shinylive with `shinylive::export("app-name", "site")`. Then deploy to a hosting site like Github or Netlify
- Embed Shinylive apps in Quarto sites, blogs, etc

filters:
- shinylive

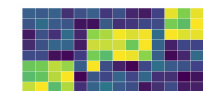
An embedded Shinylive app:

```
```{shinylive-r}
#| standalone: true
[App.py code here...]
```
```

To embed a Shinylive app in a Quarto doc, include the bold syntax.

Outputs

Reactively render R outputs



| Species | Island | Bill Length (mm) | Body Mass (g) |
|-----------|--------|------------------|---------------|
| Chonstrap | Dream | 45.70 | 3650 |
| Chonstrap | Dream | 55.80 | 4000 |
| Chonstrap | Dream | 43.50 | 3400 |
| Chonstrap | Dream | 49.60 | 3775 |

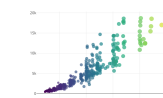
| area | peri | shape | perim |
|------|------|---------|-----------|
| 1 | 4990 | 2791.90 | 0.0903296 |
| 2 | 7862 | 3892.68 | 0.1486228 |
| 3 | 7558 | 3938.66 | 0.1833128 |
| 4 | 7352 | 3869.32 | 0.1170639 |

Current value: 30

Current value: 30



More from htmlwidgets.org ecosystem



`plotOutput(id, width, height,...)`
`renderPlot(expr, ...)`

`tableOutput(id, ...)`
`renderTable(expr, striped, ...)`

`verbatimTextOutput(id, ...)`
`renderPrint(expr, ...)`

`textOutput(id, ...)`
`renderText(expr, ...)`

`uiOutput(id, ...)`
`renderUI(expr, ...)`

`imageOutput(id, ...)`
`renderImage(expr, ...)`

`leafletOutput(id, ...)`
`renderLeaflet(expr, ...)`

`plotlyOutput(id, ...)`
`renderPlotly(expr, ...)`

See output gallery at shiny.posit.co/r/components

Inputs

Collect values from the user.

Access the current value of an input object with `input$<id>`. Input values are **reactive**.

Action `actionButton(id, label, ...)`

Action `actionLink(id, label, ...)`

☒ Choice 1
☒ Choice 2
☐ Choice 3
`checkboxGroupInput(id, label, choices, selected, ...)`

☒ Check me
`checkboxInput(id, label, value, ...)`

`dateInput(id, label, value, ...)`

`dateRangeInput(id, label, start, end, ...)`

Choose File `fileInput(id, label, ...)`

`numericInput(id, label, value, ...)`

☐ Option 1
☒ Option 2
☐ Option 3
`radioButtons(id, label, choices, selected, ...)`

`selectInput(id, label, choices, selected, multiple, ...)`
Also `selectizeInput()`

`sliderInput(id, label, min, max, value, ...)`

`textInput(id, label, value, ...)`
Also `textAreaInput()`

More from the **bslib** package:

☒ `input_dark_mode(id, mode)`

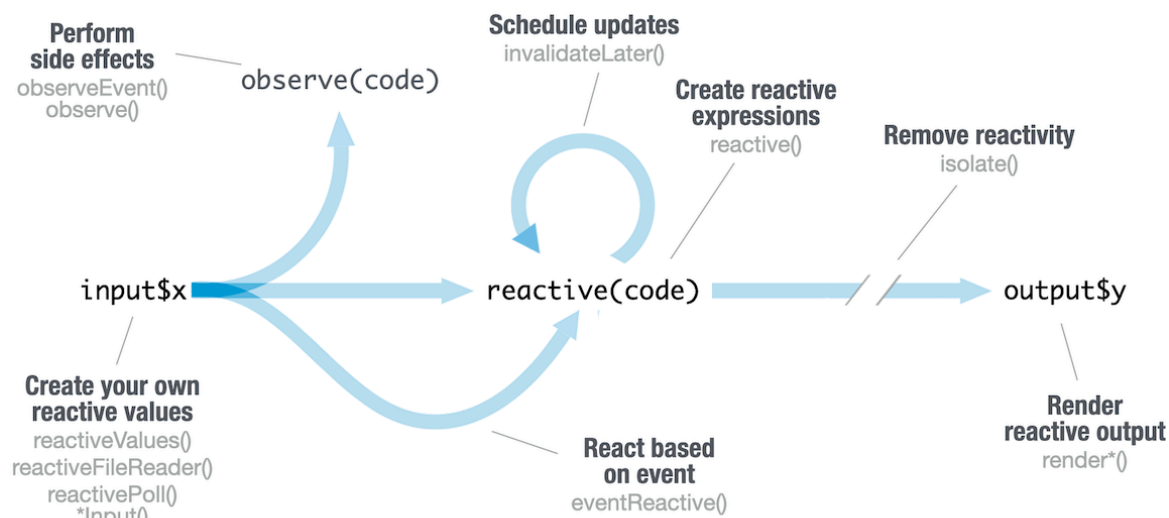
☐ `input_switch(id, label, value, ...)`

`input_task_button(id, label, value, ...)`

See input gallery at shiny.posit.co/r/components

Reactivity

Reactive values work together with reactive functions. Call a reactive value from within the arguments of one of these functions to avoid the error **Operation not allowed without an active reactive context**.



CREATE REACTIVE VALUES

```
ui <- bslib::page_fluid(
  textInput("a", "", "A")
)

server <- \(input, output){
  print(isolate(input$a))

  rv <- reactiveVal(NULL)
  print(isolate(rv()))
}

shinyApp(ui, server)
```

*Input() functions

Create a reactive value **input\$<id>** from user input.

reactiveVal(value)

Create a reactive value from a given value. Useful for managing state.

RENDER REACTIVE OUTPUT

```
ui <- bslib::page_fluid(
  textInput("a", "", "A"),
  textOutput("b")
)

server <- \(input, output){
  output$b <- renderText({
    input$a
  })
}

shinyApp(ui, server)
```

render*() functions

Produces results for a corresponding ***Output()** UI container. Re-render occurs when reactive dependencies change.

Save the results to **output\$<id>**.

CREATE REACTIVE EXPRESSIONS

```
ui <- bslib::page_fluid(
  textInput("a", "", "A"),
  textInput("z", "", "Z"),
  textOutput("b")
)

server <- \(input, output){
  re <- reactive({
    paste(input$a, input$z)
  })
  output$b <- renderText({
    re()
  })
}

shinyApp(ui, server)
```

reactive(x)

Calculate a (reactive) value based on other reactive values.

Useful for encapsulating reactive logic needed across multiple outputs.

PERFORM SIDE EFFECTS

```
ui <- bslib::page_fluid(
  textInput("a", "", "A"),
  actionButton("go", "Go")
)

server <- \(input, output){
  observe(print(input$a))

  observeEvent(input$go, {
    print(input$a)
  })
}

shinyApp(ui, server)
```

observe(x)

Observe changes to reactive values

observeEvent(eventExpr, handlerExpr)

Runs code in 2nd argument when 1st argument changes.

REACT BASED ON EVENT

```
ui <- bslib::page_fluid(
  textInput("a", "", "A"),
  actionButton("go", "Go"),
  textOutput("b")
)

server <- \(input, output){
  re <- eventReactive(
    input$go, {input$a}
  )
  output$b <- renderText({
    re()
  })
}

shinyApp(ui, server)
```

eventReactive(eventExpr, valueExpr)

Creates reactive expression with code in 2nd argument that only invalidates when reactive values in 1st argument change.

REMOVE REACTIVE DEPENDENCIES

```
ui <- bslib::page_fluid(
  textInput("a", "", "A"),
  actionButton("go", "Go"),
  textOutput("b")
)

server <- \(input, output){
  output$b <- renderText({
    input$go
    isolate(input$a)
  })
}

shinyApp(ui, server)
```

isolate(expr)

Prevent reactive values from invalidating a reactive expression.

User Interfaces (UI)

Design delightful UI with the **bslib** package. It provides layouts, components, themes, & more.

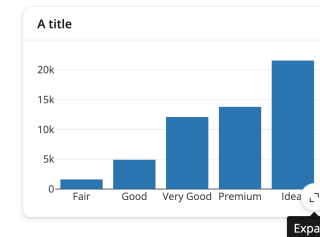
PAGE LAYOUTS

page_sidebar() Screen-filling sidebar layout
page_fillable() Screen-filling page layout
page_fixed() Constrained width page
page_fluid() Basic full-width page
page_navbar() Multi-page app with a top nav bar

CARDS

Visually group UI elements together with the **card()** component.

```
card(full_screen = T,
  card_header("A title"),
  plotOutput("my_output"),
  card_footer("A footer")
)
```



UI LAYOUTS

Multiple columns

layout_columns() Bootstrap's 12-column grid
layout_column_wrap() Equal-width columns
layout_sidebar() Resizable 2-column layout

Multiple panels

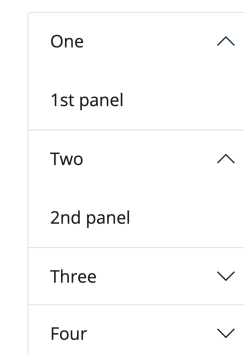
```
navset_card_underline(
  nav_panel("One", "1st panel"),
  nav_panel("Two", "2nd panel"),
  nav_menu("Menu",
    nav_panel("3", "3rd")
  )
)
```

Navigate a set of **nav_panel()**s in various ways with **navset_card_***



ACCORDIONS

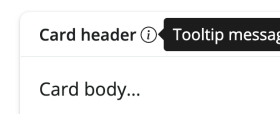
```
accordion(
  open = c("One", "Two"),
  accordion_panel("One", "1st panel"),
  accordion_panel("Two", "2nd panel"),
  accordion_panel("Three", "3rd panel"),
)
```



Tip: place within **sidebar()** to group similar inputs

TOOLTIPS

```
tooltip(
  icon("info-circle"),
  "Tooltip message"
)
```



VALUE BOXES

```
value_box(
  "Title",
  "Value",
  showcase=icon()
)
```



Custom UI

Shiny

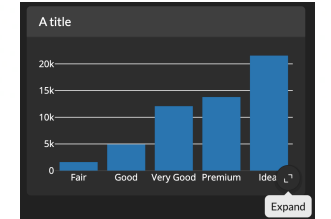
Custom how your app looks and behaves.

THEMES

Bootstrap

Choose from over a dozen pre-packaged themes

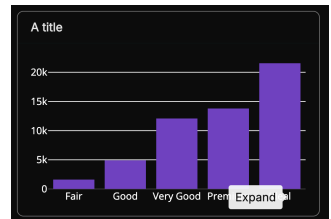
```
library(bslib)
theme <- bs_theme(
  bootswatch="darkly"
)
ui <- page_fluid(
  theme=theme, ...
)
```



Custom themes

Quickly change main colors and fonts. Change in real-time by adding **bs_themer()** to your UI.

```
bs_theme(
  bg = "#222",
  fg = "white",
  primary = "purple",
  base_font =
    font_google("Inter")
)
```



CUSTOM HTML

Shiny UI is powered by HTML, CSS, and JS:

```
page_fluid(class = "pt-3")
#> <div class="container-fluid pt-3"></div>
```

If you know these web technologies, you can customize UI to your heart's content. Start small by modifying/authoring HTML and including CSS/JS snippets. Or, go fully custom with **htmlTemplate()**



Add HTML elements with **tags**, a list of functions that parallel common HTML tags, e.g. **tags\$a()**. Unnamed arguments are treated as children and named arguments become HTML attributes.



To include a CSS file, use **includeCSS()**, or
1. Place the file in the **www** subdirectory
2. Link to it with:

```
tags$head(tags$link(href="<file name>",
  rel="stylesheet"))
```



To include JS, use **includeScript()** or
1. Place the file in the **www** subdirectory
2. Link to it with:

```
tags$head(tags$script(src="<file name>"))
```

IMAGE

To include an image:
1. Place the file in the **www** subdirectory
2. Link to it with **img(src="<file name>")**